Inferência Bayesiana II

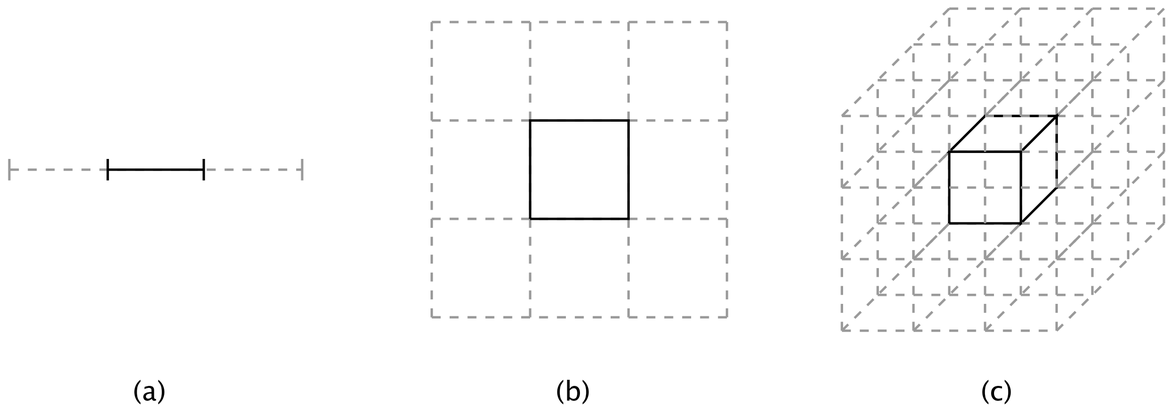
Ajuste de modelos, Aproximação quadrática, MCMC

Diogo Melo - 14/10/2026

Como obter amostras da posteriori?

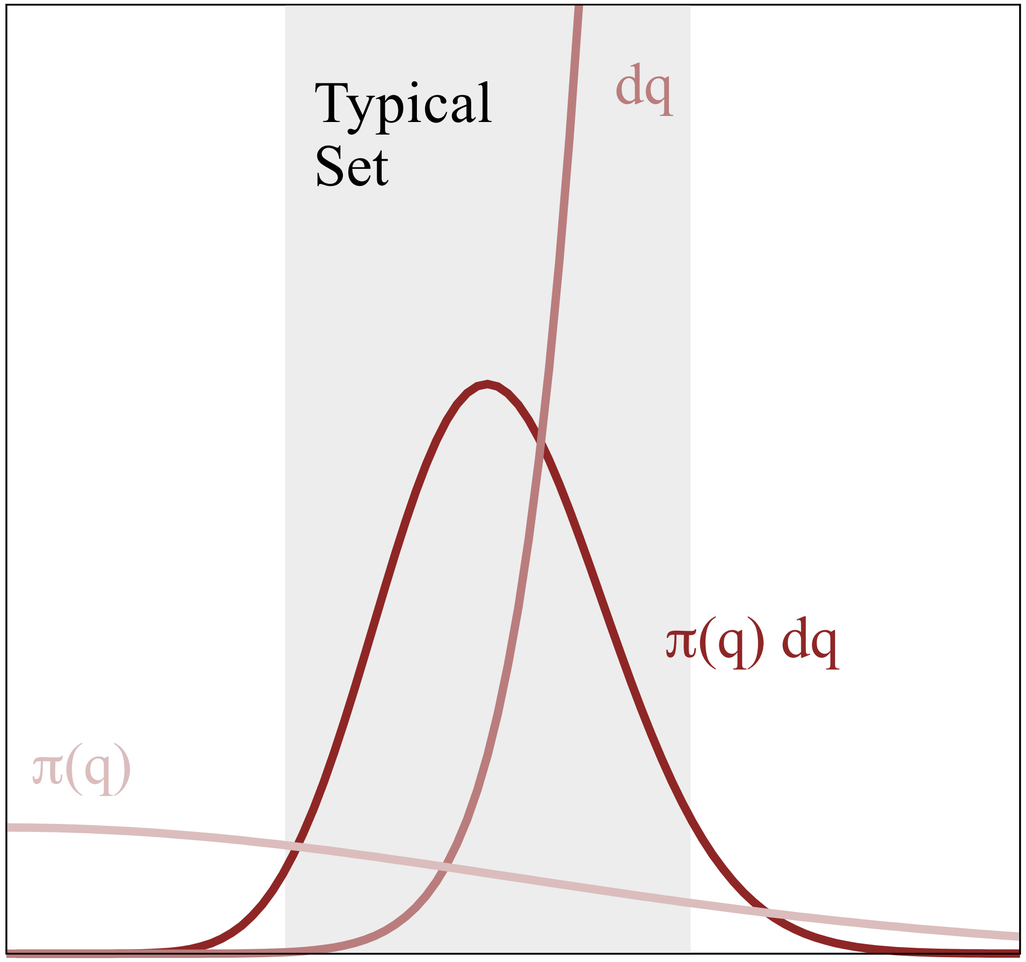
- Não existem métodos analíticos para achar as regiões de probabilidade alta de distribuições.
- Portanto, a maioria dos modelos é ajustado usando métodos computacionais.
- Para modelos com poucos parâmetros, podemos usar uma busca exaustiva para achar as regiões de probabilidade alta.

- Esse método deixa de funcionar quando o número de parâmetros aumenta.



Betancourt, M. A Conceptual Introduction to Hamiltonian Monte Carlo. *arXiv [stat.ME]* (2017)

Conjunto típico



Betancourt, M. A Conceptual Introduction to Hamiltonian Monte Carlo. *arXiv [stat.ME]* (2017)

Achando o conjunto típico

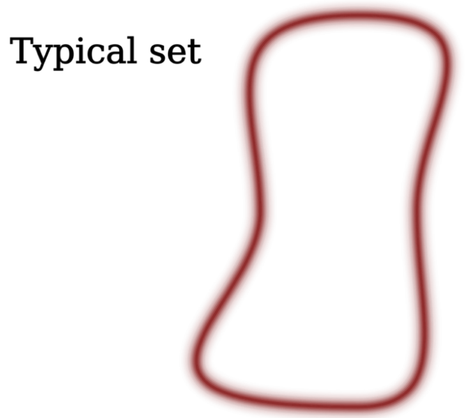
Markov Chain Monte Carlo (MCMC) Method

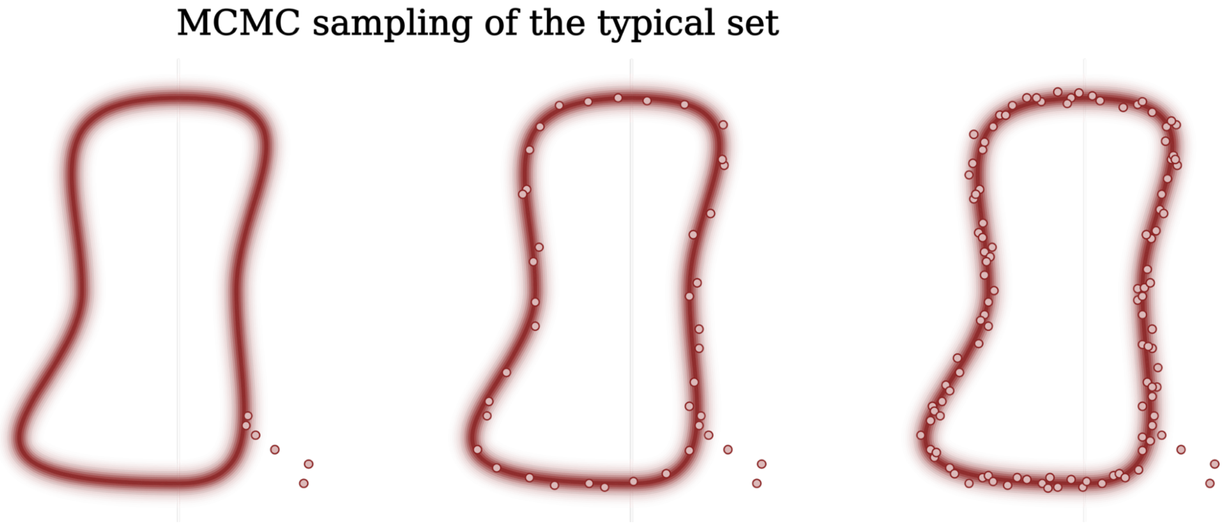
Ache uma sequência de pontos no espaço de parâmetro:

$$\theta_1 \rightarrow \theta_2 \rightarrow \theta_3 \rightarrow \theta_4 \rightarrow \cdots$$

Tal que:

$$\{\theta_1, \cdots, \theta_n\} \sim P(\theta \mid y)$$





MCMC samplers

- Metropolis–Hastings algorithms (broad class of samplers, very general).
  - Most methods in the wild are some flavor of this.
- Reversible Jump MCMC (used in many phylogenetic packages).
  - Allows for posterior distributions with variable dimensionality.
- Usable non-mcmc methods: R-INLA - integrated nested Laplace approximation.
  - Great for structural equation modeling, much faster for some classes of models.

MCMC samplers

- Metropolis–Hastings algorithms (broad class of samplers, very general).
  - Most methods in the wild are some flavor of this.
- Reversible Jump MCMC (used in many phylogenetic packages).
  - Allows for posterior distributions with variable dimensionality.
- Usable non-mcmc methods: R-INLA - integrated nested Laplace approximation.
  - Great for structural equation modeling, much faster for some classes of models.
- Gibbs samplers.
  - Mostly surpassed, but still in wide use.
  - Can sample discrete parameters.
  - Requires particular types of priors.
  - Software: WinBugs, Bugs, Jags...
- Hamiltonian Monte Carlo samplers
  - Discrete parameters must be integrated.
  - Can fit dynamic models using differential equations.
  - Software: PyMC3, Edward, Stan (rethinking engine)...

Qual a diferença entre esses amostradores?

Basicamente, é a distribuição de probabilidade da proposta de transição:

$$P(\theta_{i+1} \mid \theta_i)$$

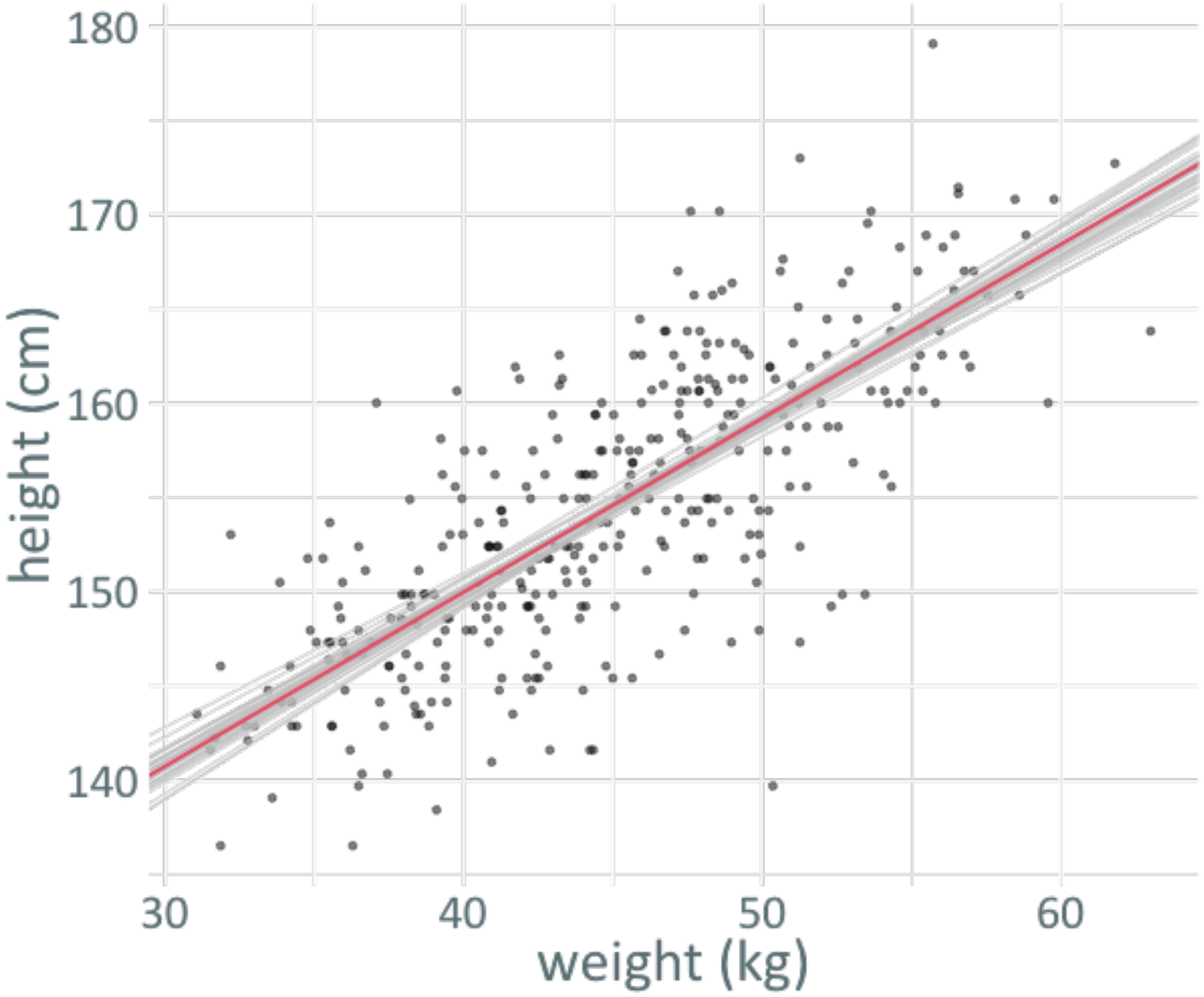
$$\theta_1 \rightarrow \theta_2 \rightarrow \theta_3 \rightarrow \theta_4 \rightarrow \cdots$$

Vamos explorar isso usando uma visualização:

https://chi-feng.github.io/mcmc-demo/app.html

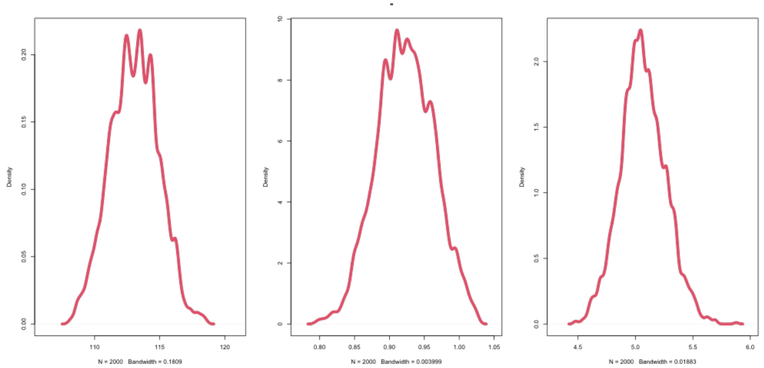
Modelo altura-peso

$$y_i \sim Normal(\mu_i, \sigma)$$
$$\mu_i = \alpha + \beta x_i$$
$$\alpha \sim Normal(0, 20)$$
$$\beta \sim lognormal(0, 1)$$
$$\sigma \sim Exponential(1)$$



Amostradores no Rethinking

`quap()` – aproximação quadrática



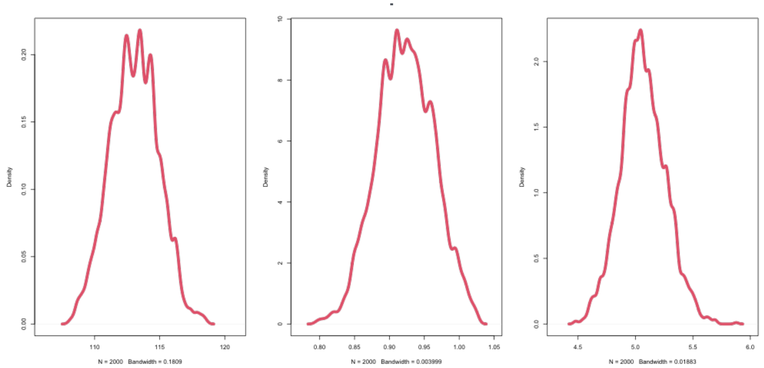
$$y_i \sim Normal(\mu_i, \sigma)$$
$$\mu_i = \alpha + \beta x_i$$
$$\alpha \sim Normal(0, 20)$$
$$\beta \sim lognormal(0, 1)$$
$$\sigma \sim Exponential(1)$$

```
# Data
library(rethinking)
d2 <- Howell1[ Howell1$age >= 18 , ]

# Model
quap(alist(
  y ~ dnorm(mu, sigma),
  mu <- a + b * x,
  a ~ dnorm(0, 20),
  b ~ dlnorm(0, 1),
  sigma ~ dexp(1)),
  data = list(y = d2$height,
              x = d2$weight))
```

Amostradores no Rethinking

`quap()` – aproximação quadrática



$$y_i \sim Normal(\mu_i, \sigma)$$
$$\mu_i = \alpha + \beta x_i$$
$$\alpha \sim Normal(0, 20)$$
$$\beta \sim lognormal(0, 1)$$
$$\sigma \sim Exponential(1)$$

```
# Data
library(rethinking)
d2 <- Howell1[ Howell1$age >= 18 , ]

# Model
quap(alist(
  y ~ dnorm(mu, sigma),
  mu <- a + b * x,
  a ~ dnorm(0, 20),
  b ~ dlnorm(0, 1),
  sigma ~ dexp(1)),
  data = list(y = d2$height,
              x = d2$weight))
```

Amostradores no Rethinking

`ulam()` – Hamiltonian Monte Carlo

$$
\begin{aligned}
y_i &\sim Normal(\mu_i, \sigma) \\
\mu_i &= \alpha + \beta x_i \\
\alpha &\sim Normal(0, 20) \\
\beta &\sim lognormal(0, 1) \\
\sigma &\sim Exponential(1)
\end{aligned}
$$

```
# Data
library(rethinking)
d2 <- Howell1[ Howell1$age >= 18 , ]

# Model
ulam(alist(
  y ~ normal(mu, sigma),
  mu <- a + b * x,
  a ~ normal(0, 20),
  b ~ lognormal(0, 1),
  sigma ~ exponential(1)),
  data = list(y = d2$height,
              x = d2$weight),
  iter = 1000, chains = 4, cores = 4)
```

Amostradores no Rethinking

`ulam()` – Hamiltonian Monte Carlo

$$y_i \sim Normal(\mu_i, \sigma)$$
$$\mu_i = \alpha + \beta x_i$$
$$\alpha \sim Normal(0, 20)$$
$$\beta \sim lognormal(0, 1)$$
$$\sigma \sim Exponential(1)$$

```
# Data
library(rethinking)
d2 <- Howell1[ Howell1$age >= 18 , ]

# Model
ulam(alist(
  y ~ normal(mu, sigma),
  mu <- a + b * x,
  a ~ normal(0, 20),
  b ~ lognormal(0, 1),
  sigma ~ exponential(1)),
  data = list(y = d2$height,
              x = d2$weight),
  iter = 1000, chains = 4, cores = 4)
```

Amostradores no Rethinking

`ulam()` – Hamiltonian Monte Carlo

$$y_i \sim Normal(\mu_i, \sigma)$$
$$\mu_i = \alpha + \beta x_i$$
$$\alpha \sim Normal(0, 20)$$
$$\beta \sim lognormal(0, 1)$$
$$\sigma \sim Exponential(1)$$

```
# Data
library(rethinking)
d2 <- Howell1[ Howell1$age >= 18 , ]

# Model
ulam(alist(
  y ~ normal(mu, sigma),
  mu <- a + b * x,
  a ~ normal(0, 20),
  b ~ lognormal(0, 1),
  sigma ~ exponential(1)),
  data = list(y = d2$height,
              x = d2$weight),
  iter = 1000, chains = 4, cores = 4)
```

rethinking gera código do Stan

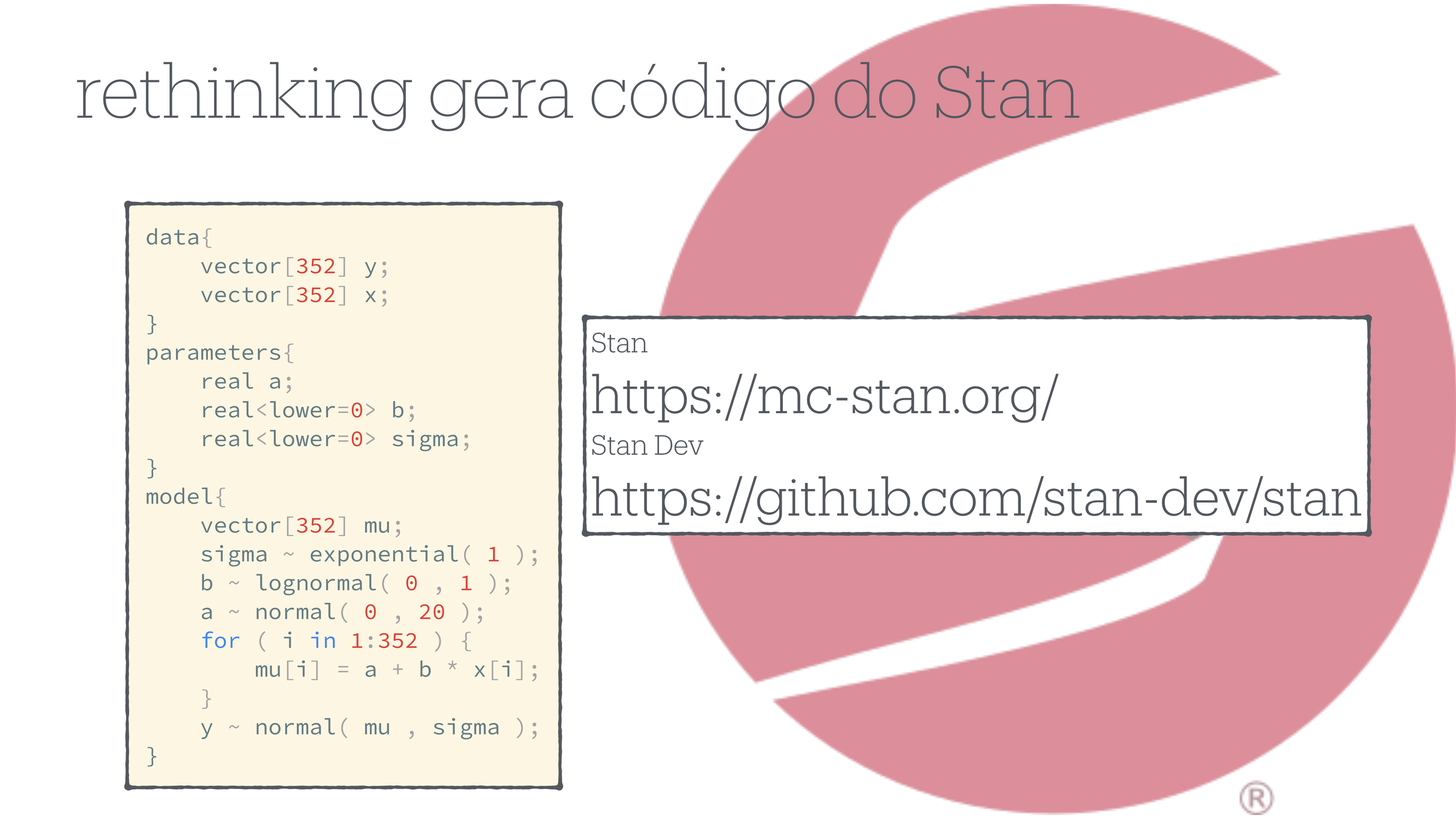
```
data{
    vector[352] y;
    vector[352] x;
}
parameters{
    real a;
    real<lower=0> b;
    real<lower=0> sigma;
}
model{
    vector[352] mu;
    sigma ~ exponential( 1 );
    b ~ lognormal( 0 , 1 );
    a ~ normal( 0 , 20 );
    for ( i in 1:352 ) {
        mu[i] = a + b * x[i];
    }
    y ~ normal( mu , sigma );
}
```

Stan

https://mc-stan.org/

Stan Dev

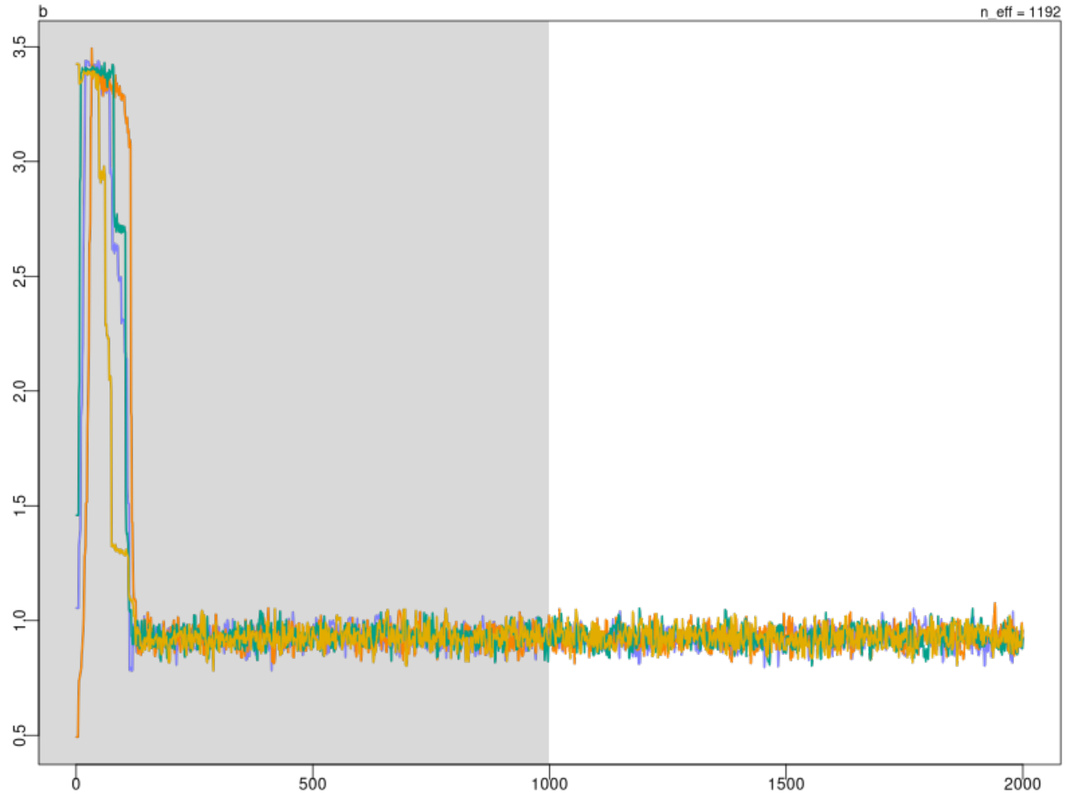
https://github.com/stan-dev/stan



Argumentos dos amostradores

- Chains: ajuste o modelo várias vezes
- Cores: em paralelo (mais rápido)
- Iterations: quantas amostras
- Warm-up (or burn-in): amostras do começo da cadeia, descartadas no final

Cadeias e convergência



Model summary

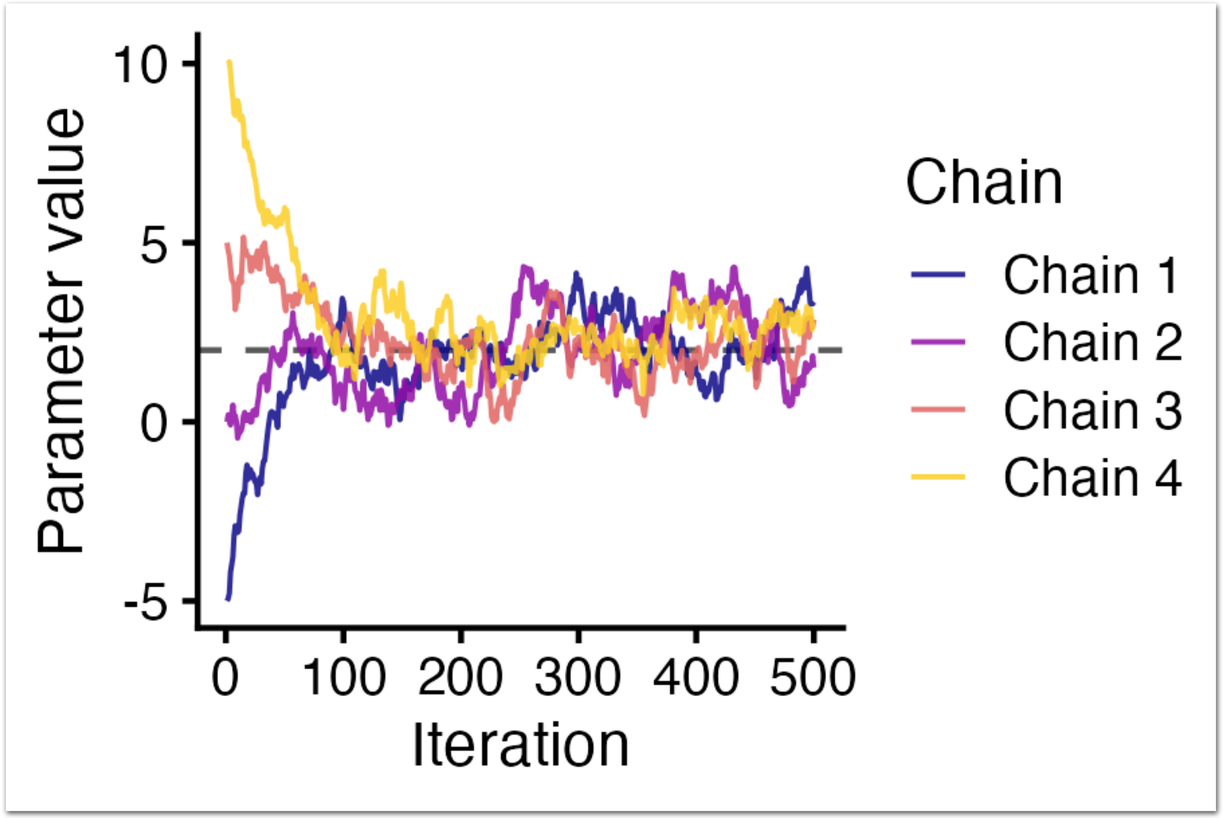
```
> precis(fit, prob = 0.95)
         mean   sd    2.5%  97.5% rhat ess_bulk
a      112.98 1.94 109.41 116.91 1.01   515.41
b        0.92 0.04   0.84   1.00 1.01   522.39
sigma    5.08 0.19   4.74   5.48 1.00   845.79
```

Aki Vehtari (2021). Comparison of MCMC effective sample size estimators. Retrieved from https://avehtari.github.io/rhat_ess/ess_comparison.html

Verificando convergência

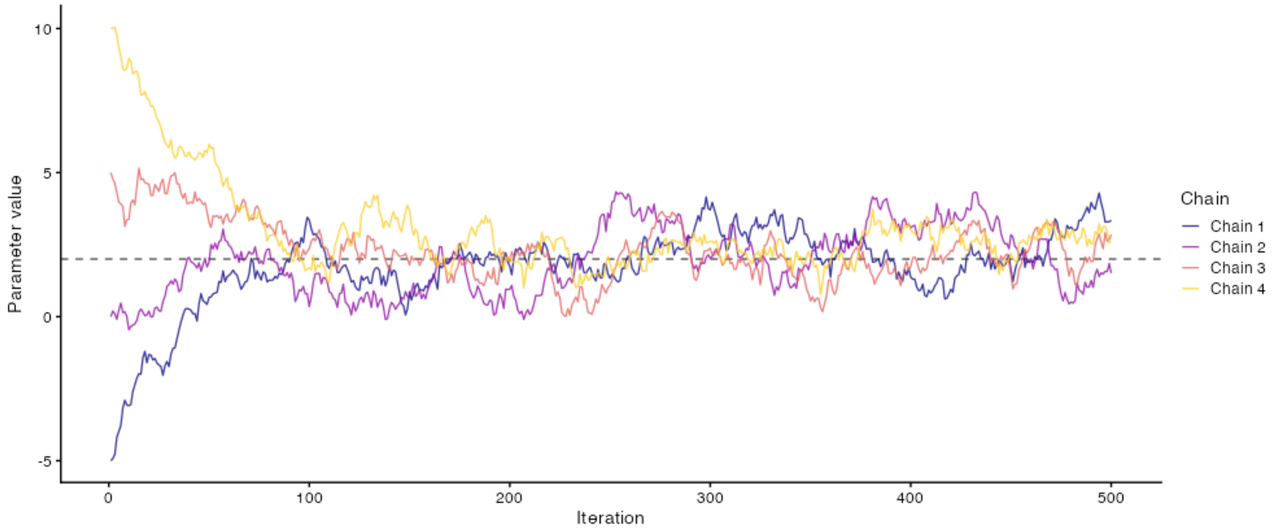
Traceplots e $\hat{R}$ (R-hat, R-chapéu)

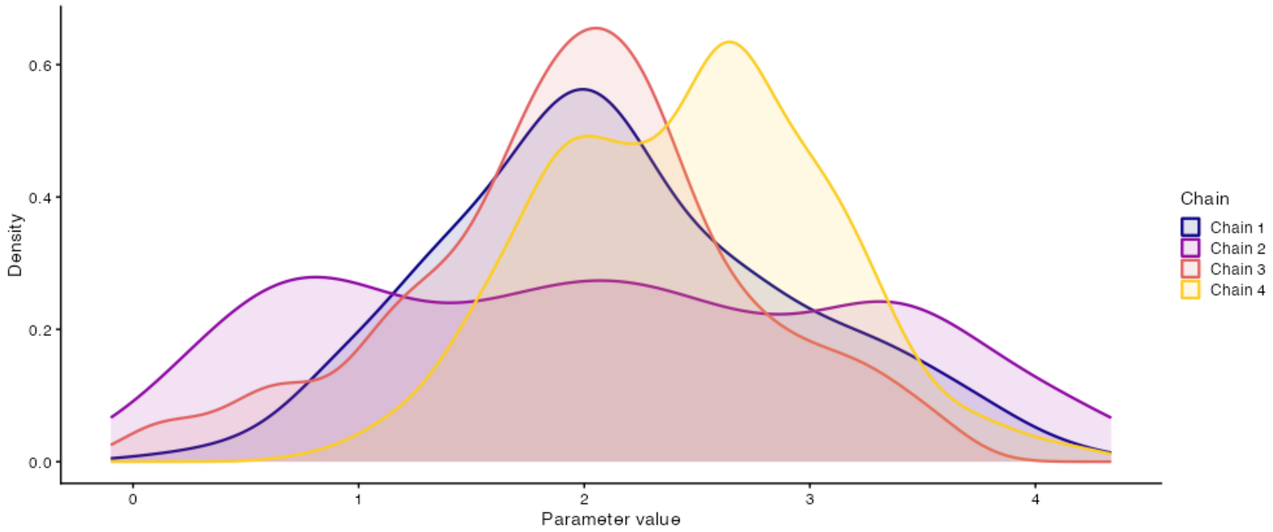
- É fundamental verificar a convergência dos modelos que ajustamos.
- Uma primeira investigação pode ser feita de forma gráfica, olhando para as amostras sequenciais que foram feitas da posteriori.
- Outra estratégia é comparar as estimativas resultantes de cadeias diferentes.
    - O R-chapéu é uma estatística feita para comparar cadeias diferentes. Ela compara a variância dentro de cadeias com a variância entre cadeias.
    - **Idealmente, $\hat{R}$ de ser próximo de 1.**



Verificando convergência

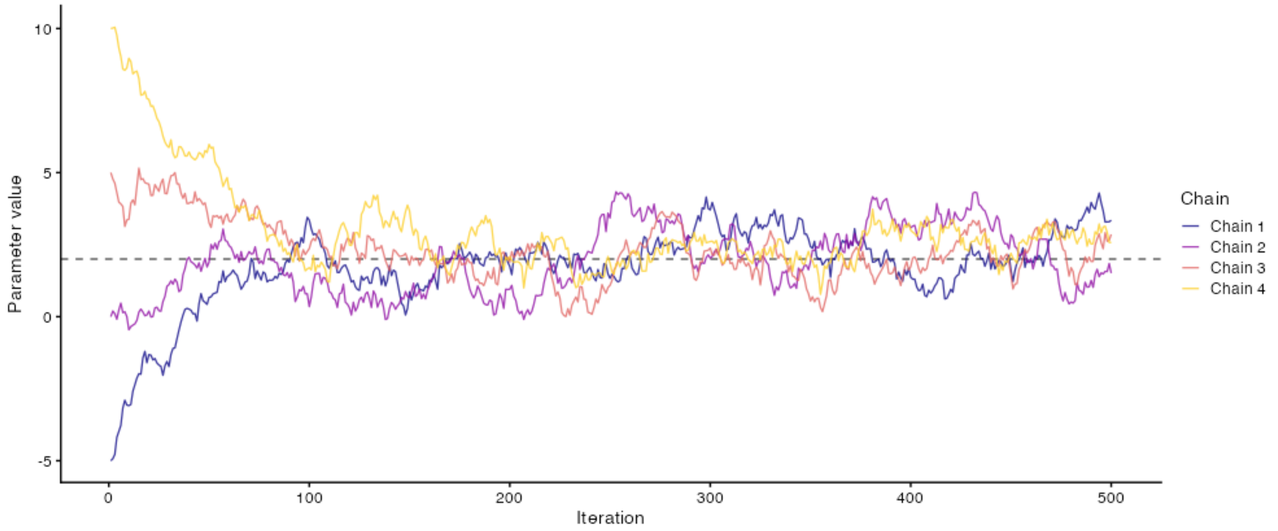
Traceplots e $\hat{R}$ (R-hat, R-chapéu)

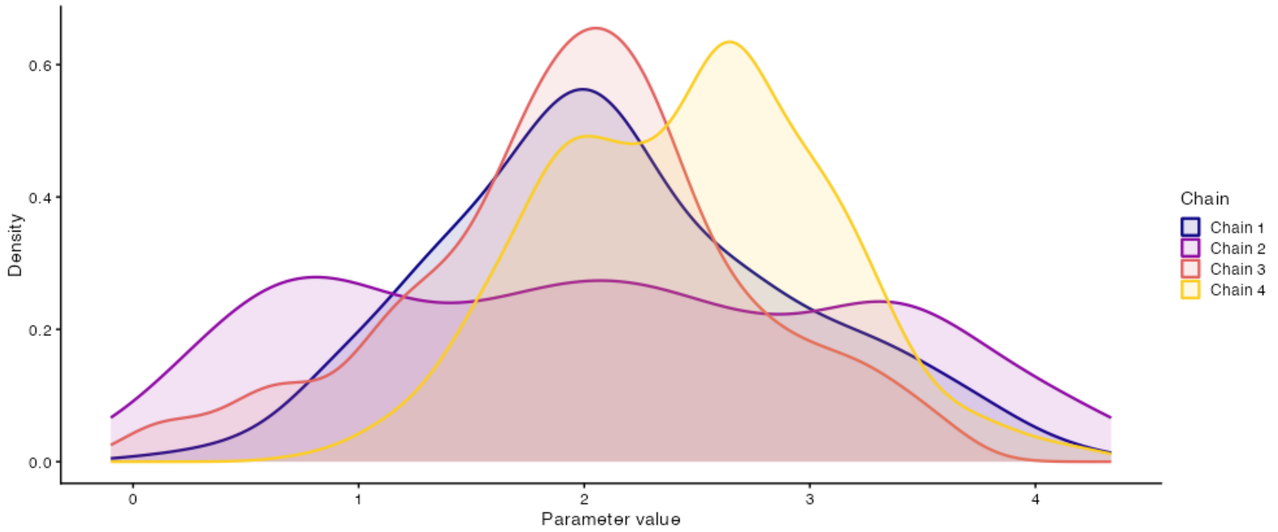


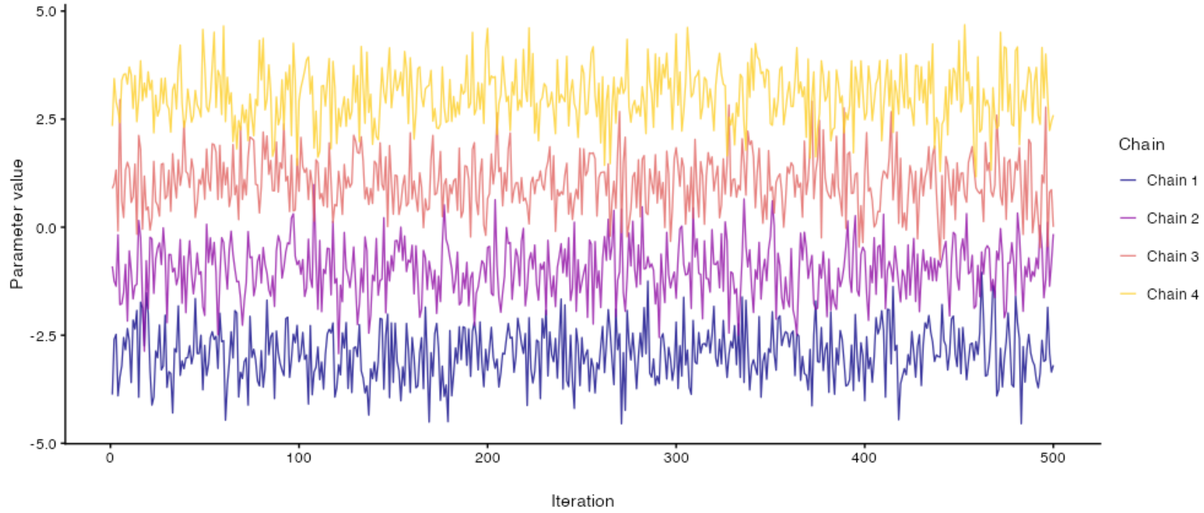


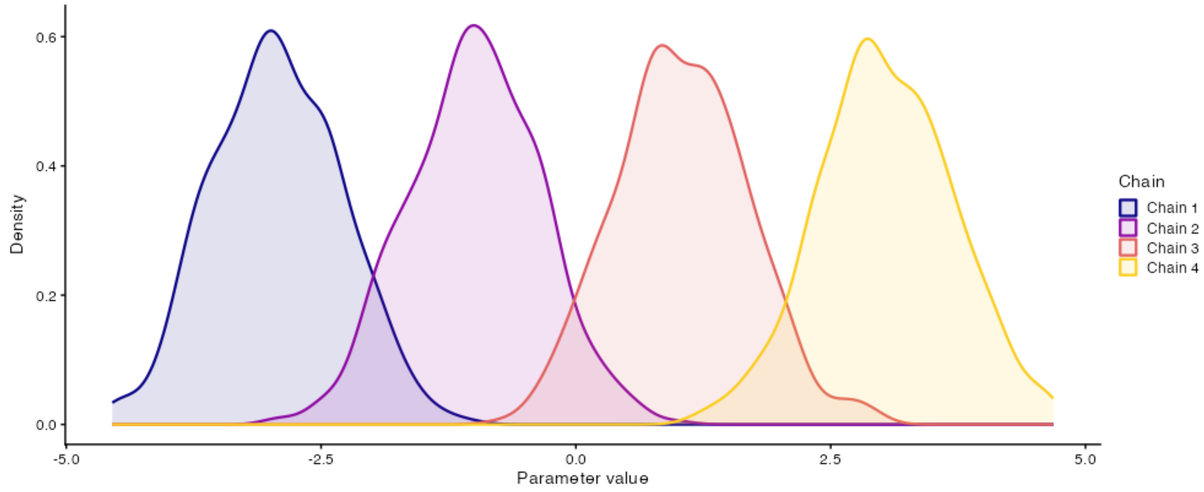
Verificando convergência

Traceplots e $\hat{R}$ (R-hat, R-chapéu)



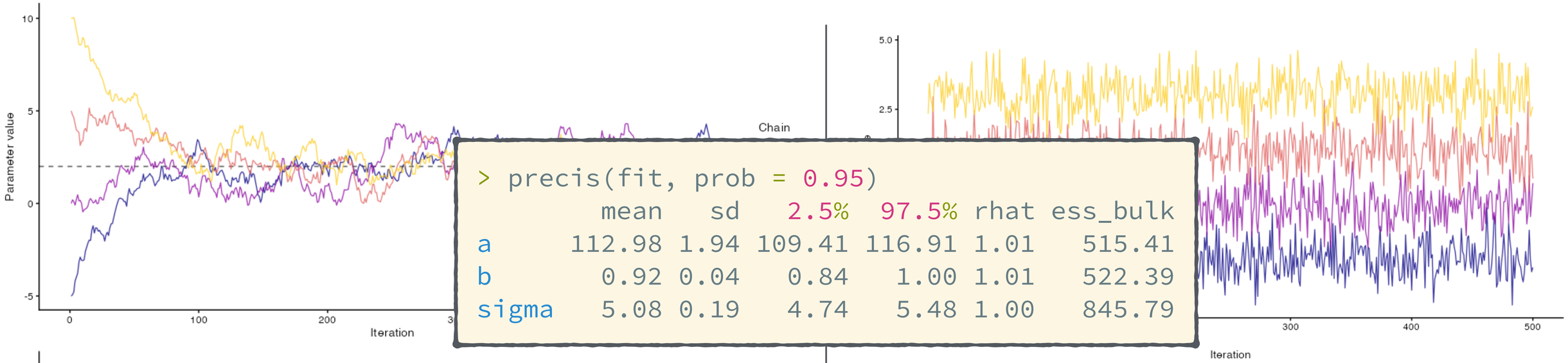




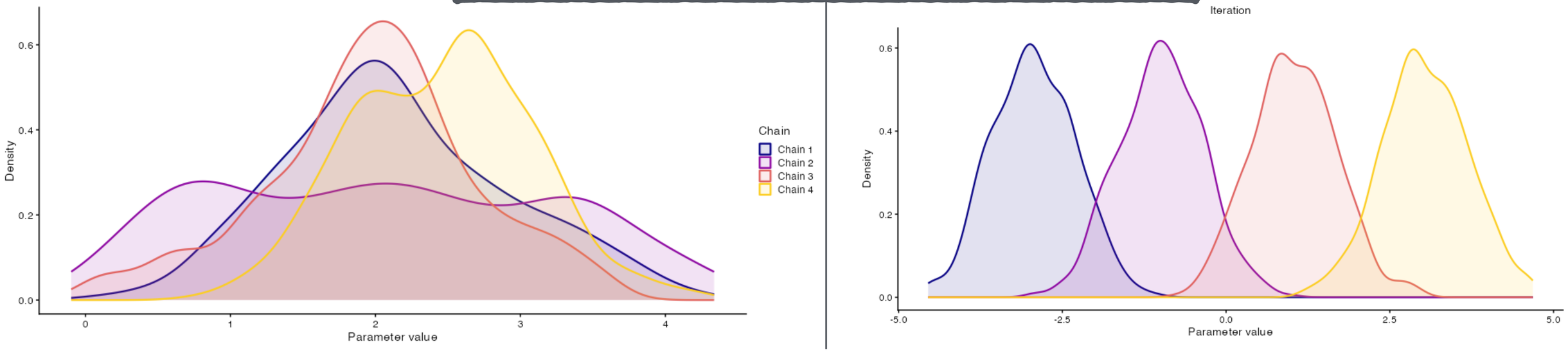


Verificando convergência

Traceplots e $\hat{R}$ (R-hat, R-chapéu)




```
> precis(fit, prob = 0.95)
          mean   sd    2.5%  97.5% rhat ess_bulk
a       112.98 1.94 109.41 116.91 1.01   515.41
b         0.92 0.04   0.84   1.00 1.01   522.39
sigma     5.08 0.19   4.74   5.48 1.00   845.79
```



Dependencia entre parâmetros

Correlações na posteriori

- Em muitos modelos, dependência entre parâmetros podem ser indicadores de problemas.
- Uma ferramenta para investigar esse tipo de questão é o pairs plot.
- São gráficos par a par da distribuição a posteriori dos parâmetros.

```
# Data
library(rethinking)
d2 <- Howell1[ Howell1$age >= 18 , ]

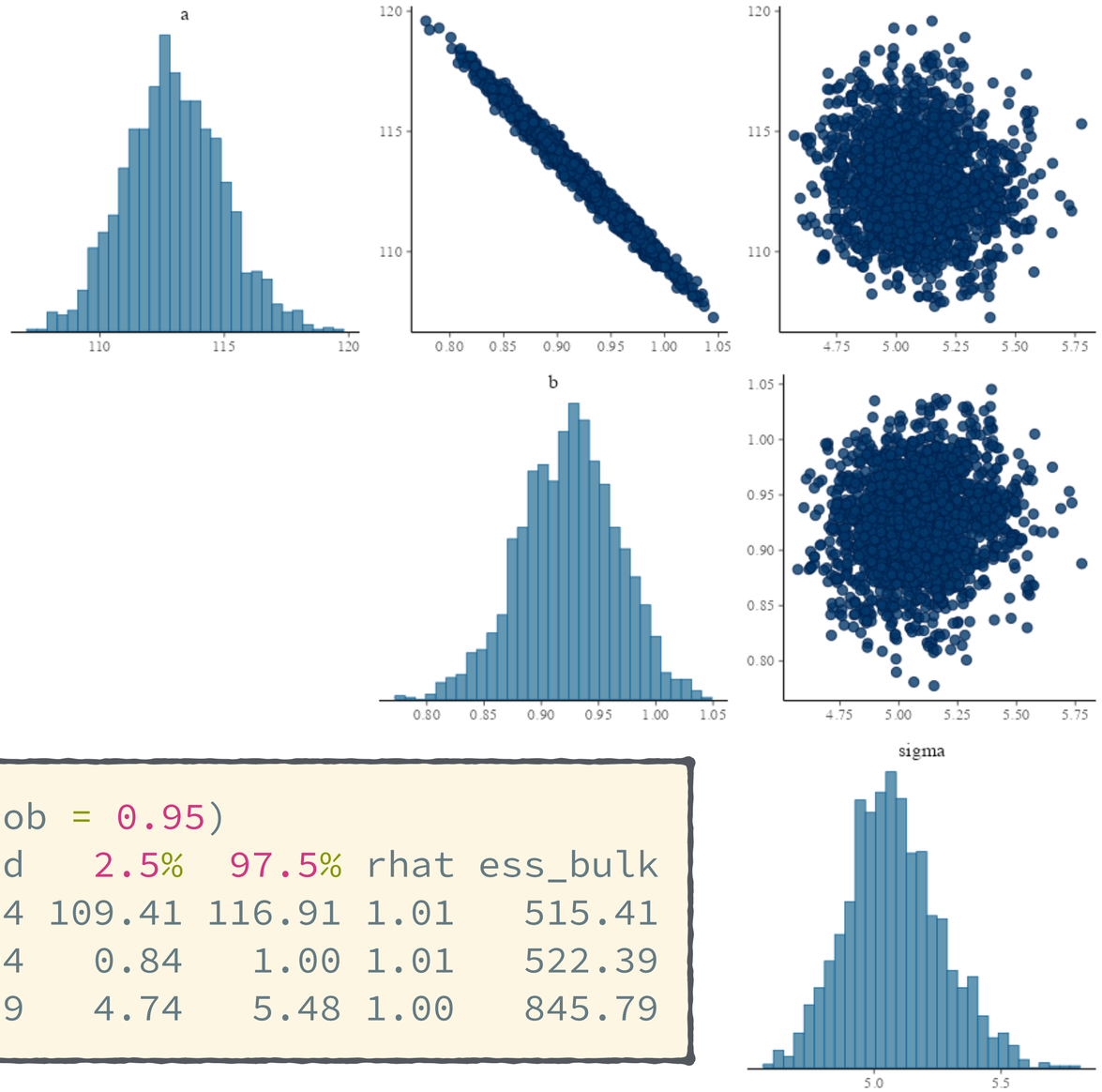
# Model
ulam(alist(
  y ~ normal(mu, sigma),
  mu <- a + b * x,
  a ~ normal(0, 20),
  b ~ lognormal(0, 1),
  sigma ~ exponential(1)),
  data = list(y = d2$height,
              x = d2$weight),
  iter = 1000, chains = 4, cores = 4)
```

Dependencia entre parâmetros

Correlações na posteriori

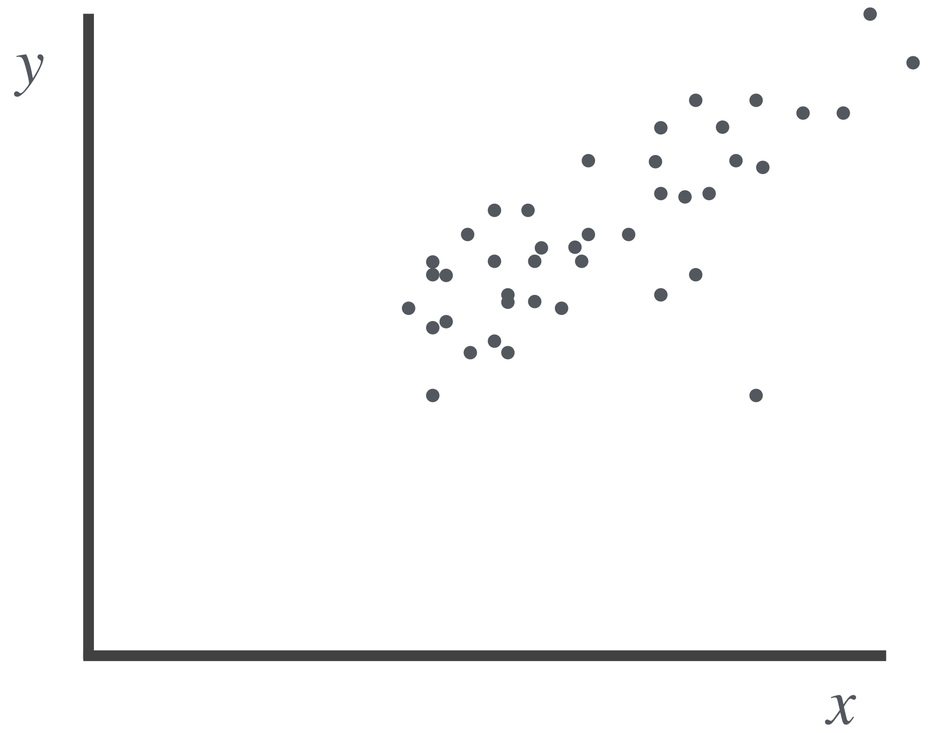
```
# Data
library(rethinking)
d2 <- Howell1[ Howell1$age >= 18 , ]

# Model
ulam(alist(
  y ~ normal(mu, sigma),
  mu <- a + b * x,
  a ~ normal(0, 20),
  b ~ lognormal(0, 1),
  sigma ~ exponential(1)),
  data = list(y = d2$height,
              x = d2$weight),
  iter = 1000, chains = 4)
```

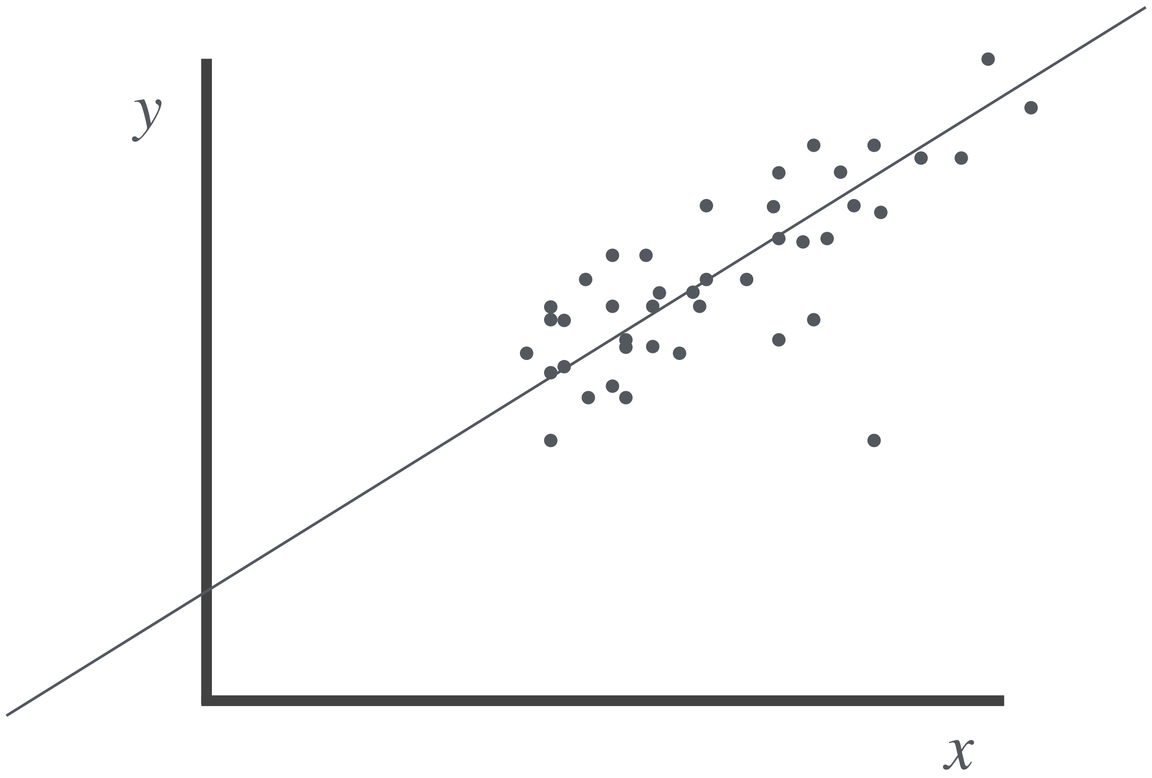



```
> precis(fit, prob = 0.95)
        mean   sd   2.5%  97.5% rhat ess_bulk
a     112.98 1.94 109.41 116.91 1.01   515.41
b       0.92 0.04   0.84   1.00 1.01   522.39
sigma   5.08 0.19   4.74   5.48 1.00   845.79
```

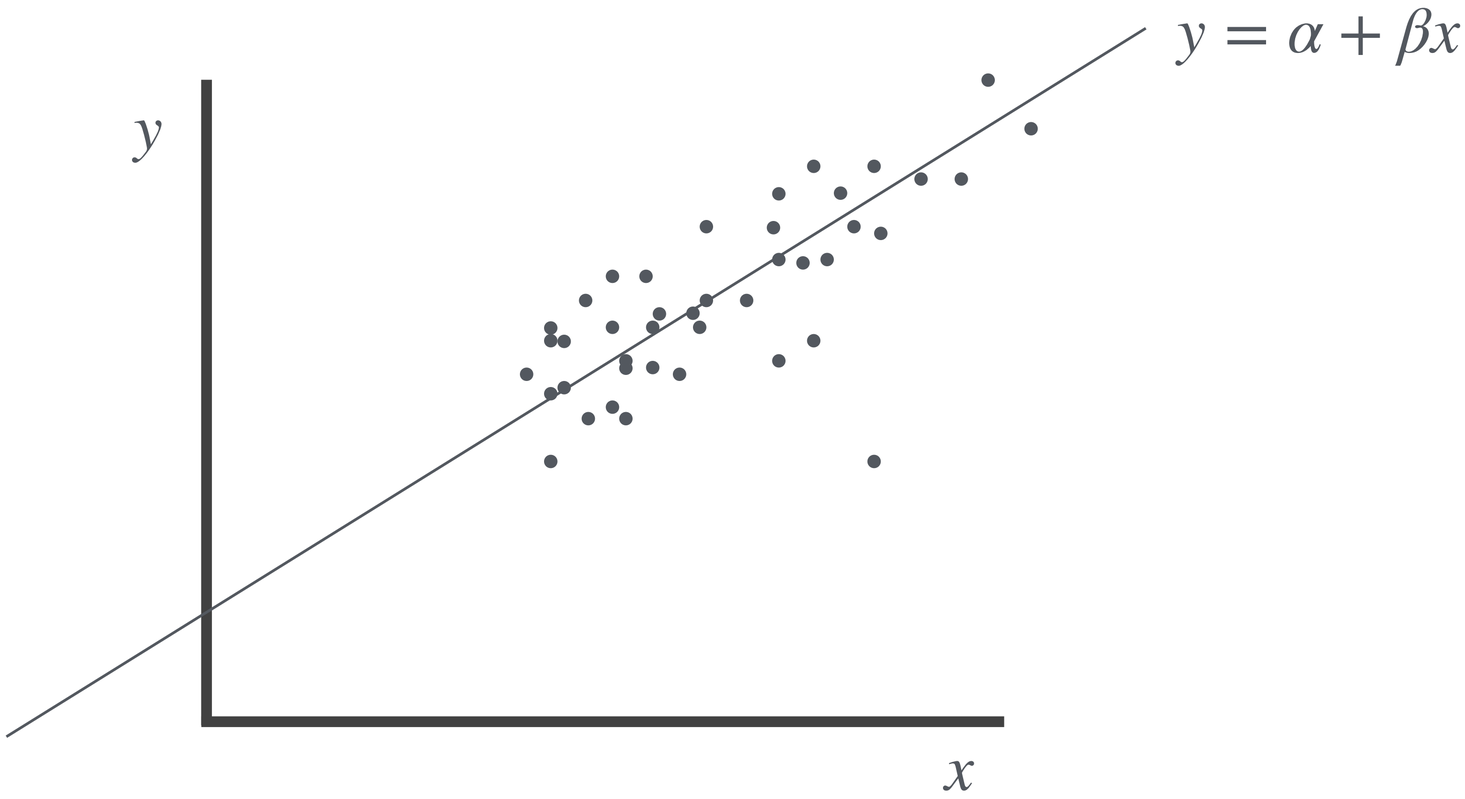
Correlações intercepto-inclinação



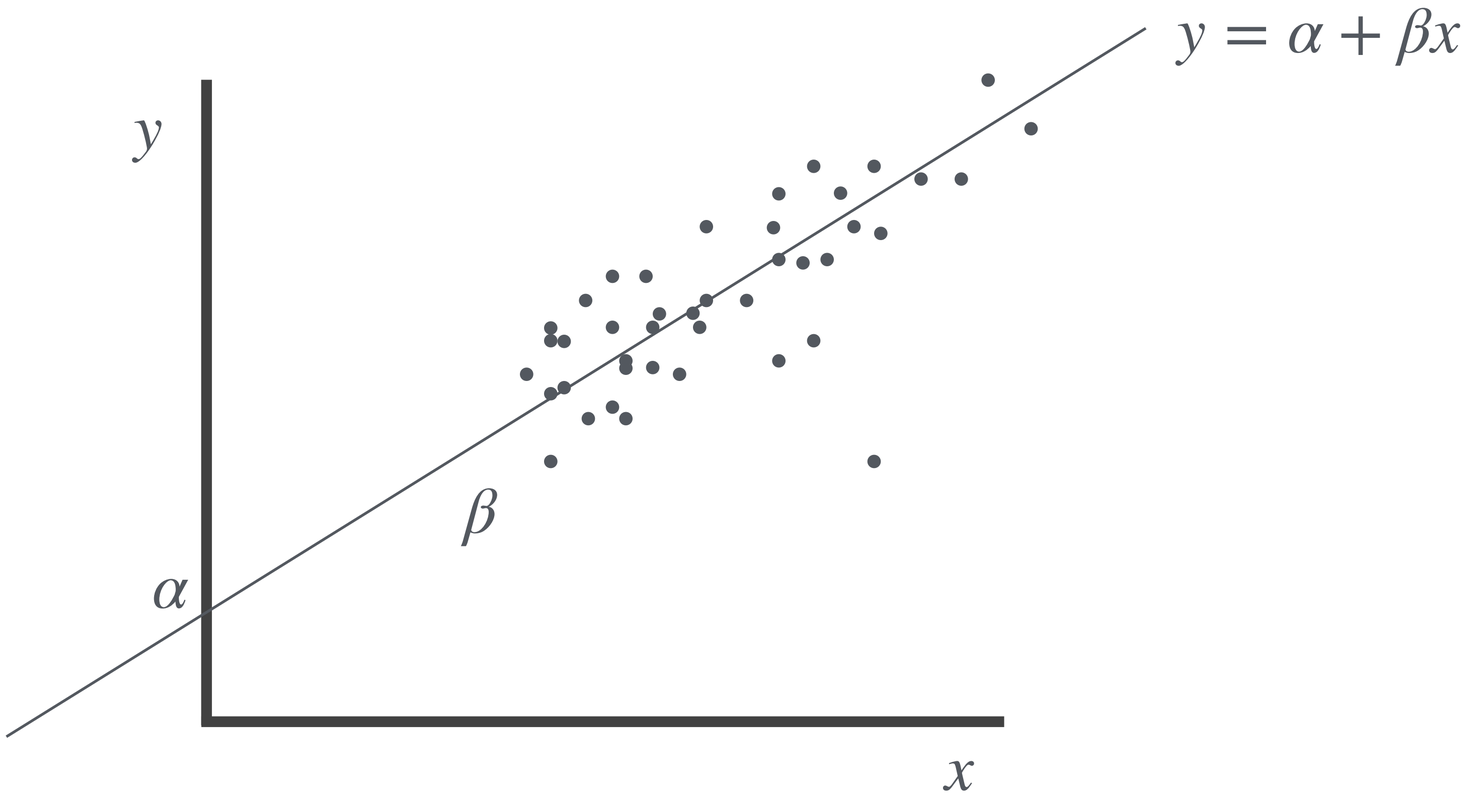
Correlações intercepto-inclinação



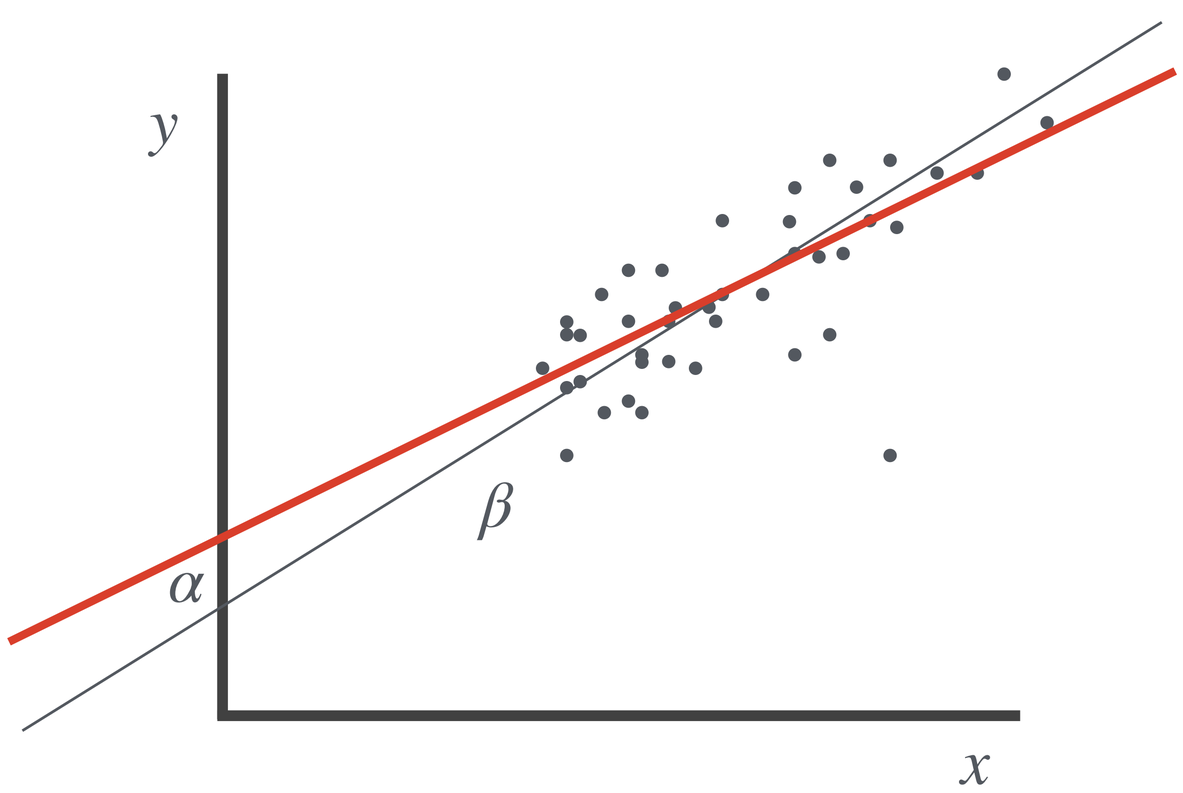
Correlações intercepto-inclinação

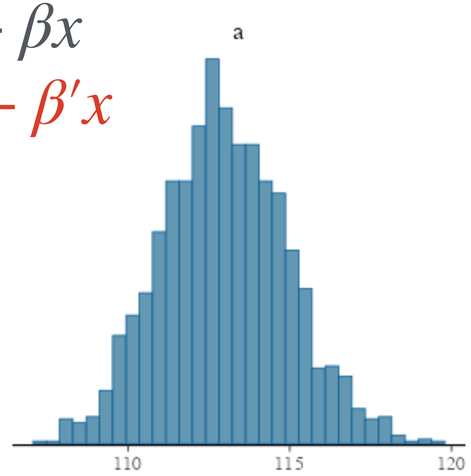


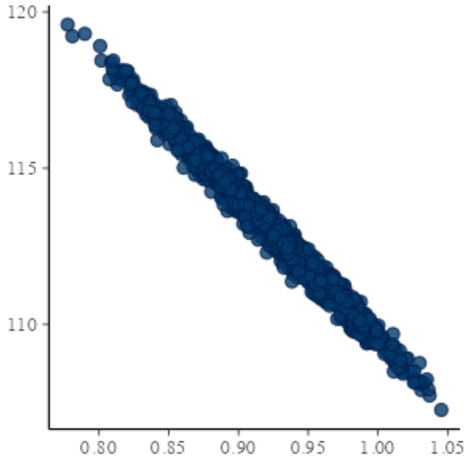
Correlações intercepto-inclinação

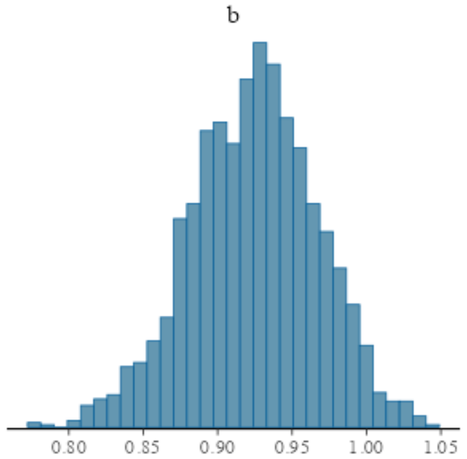


Correlações intercepto-inclinação

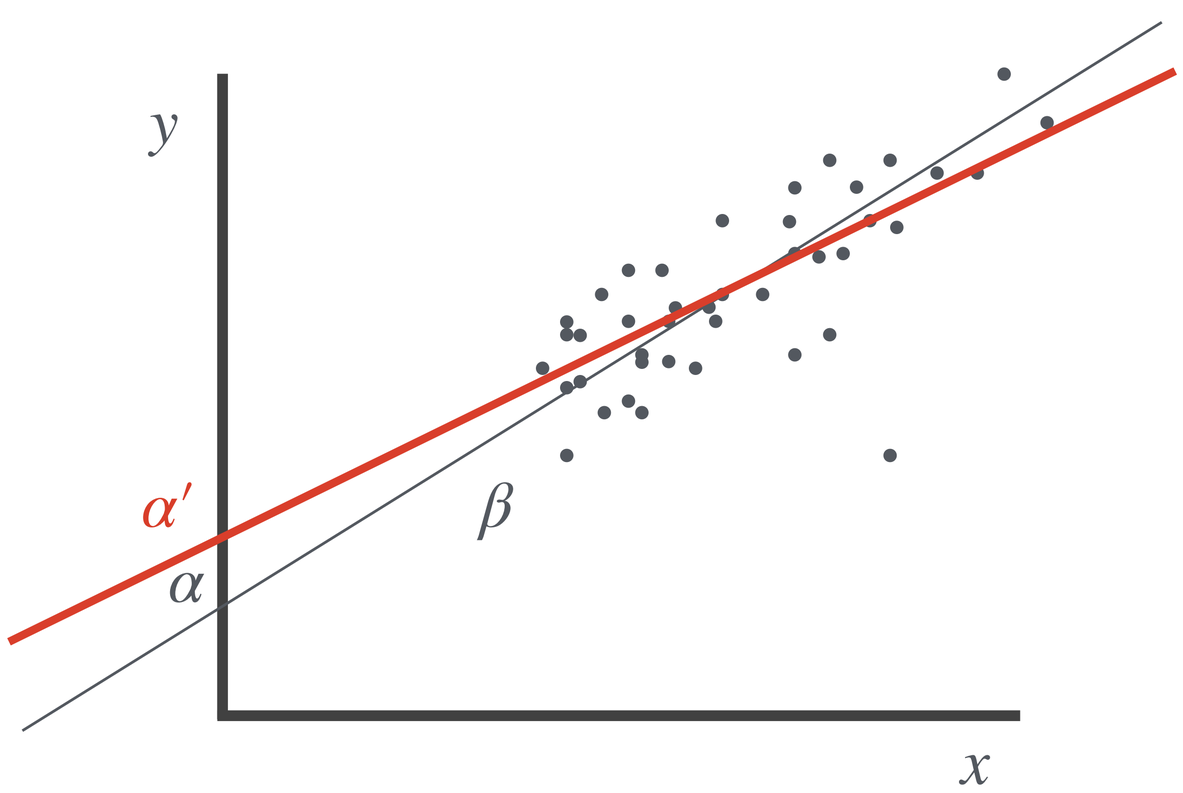


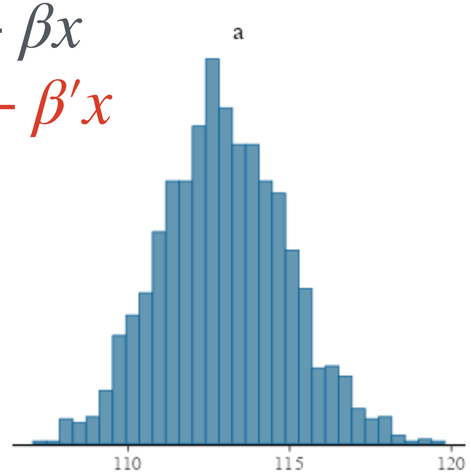


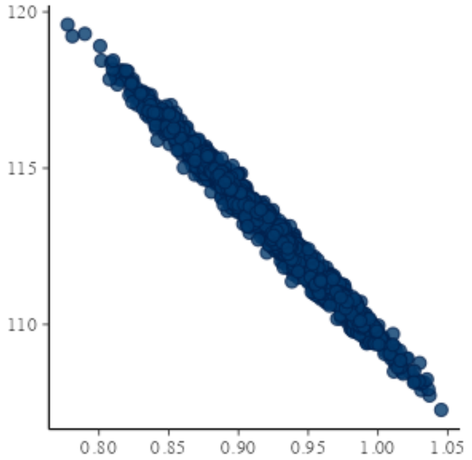


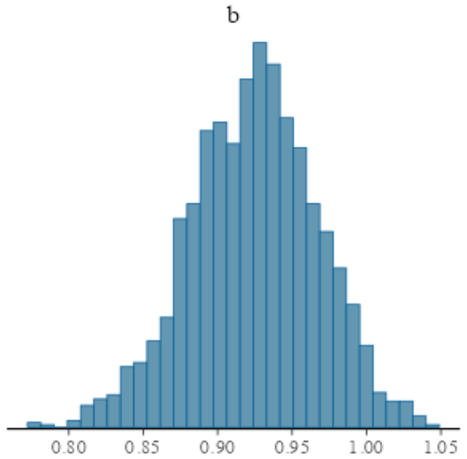


Correlações intercepto-inclinação

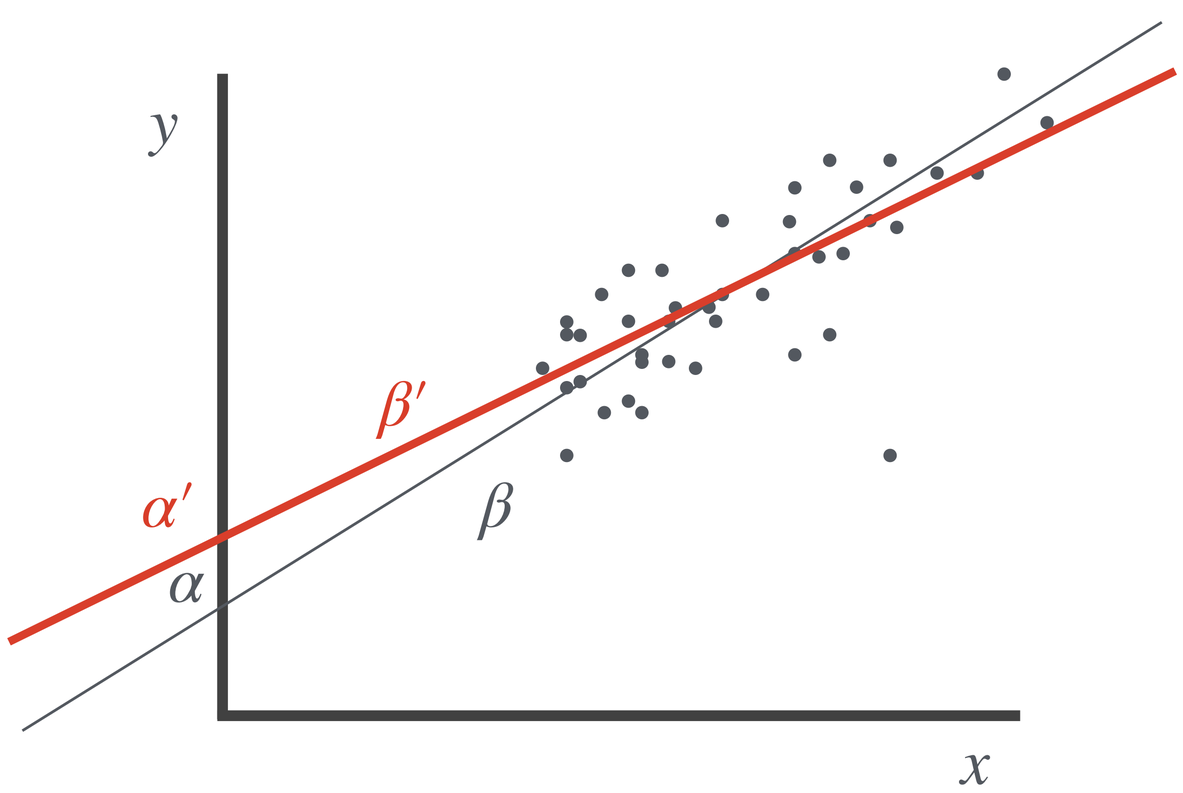


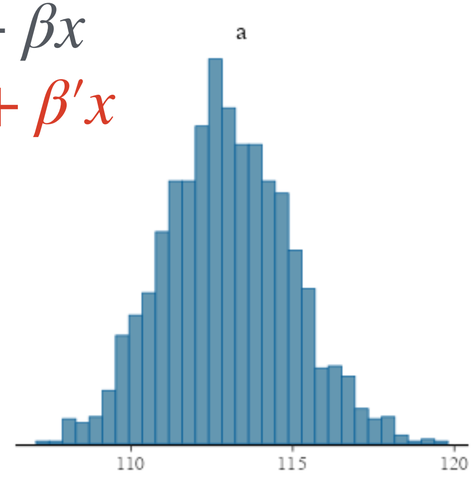


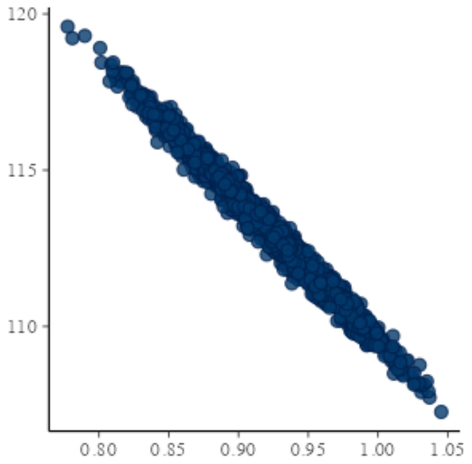


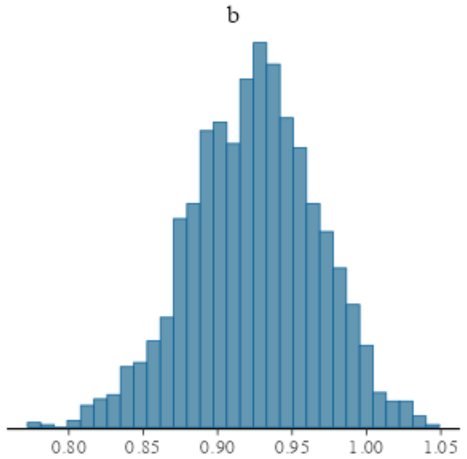


Correlações intercepto-inclinação

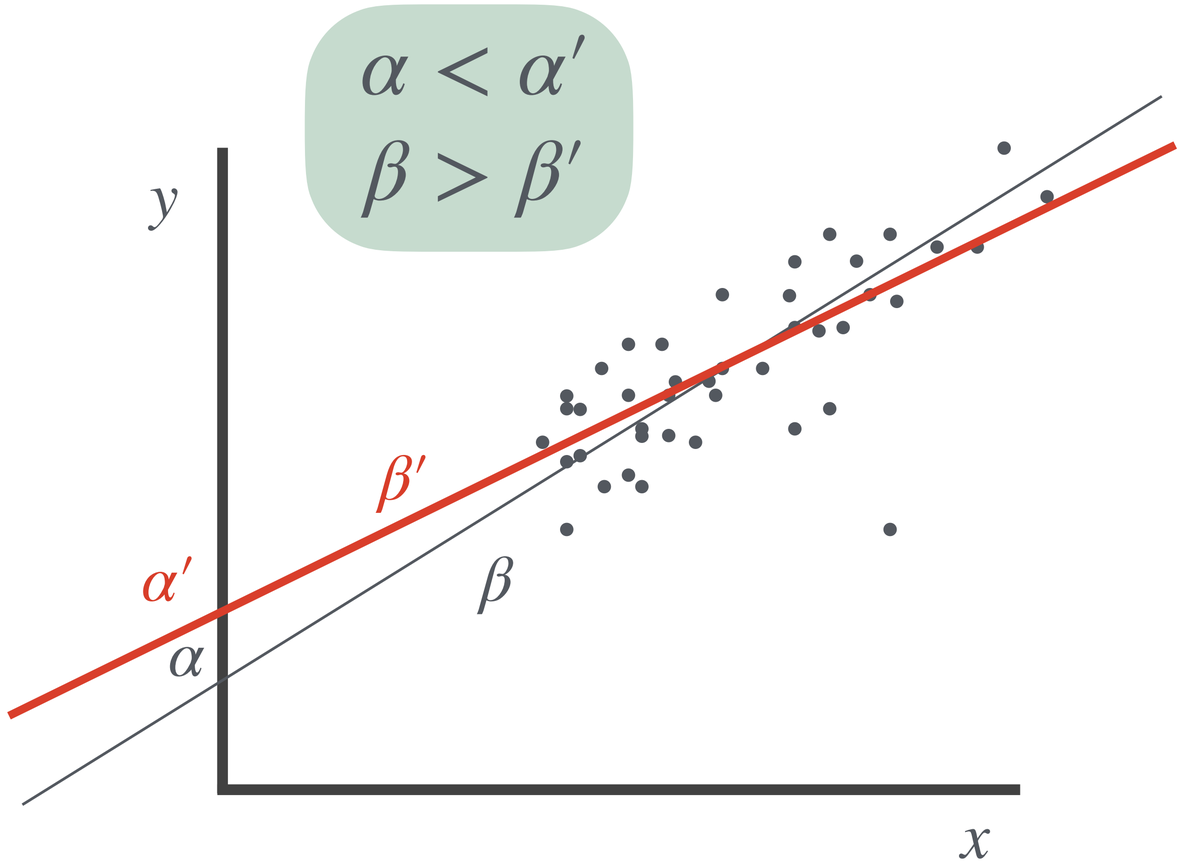


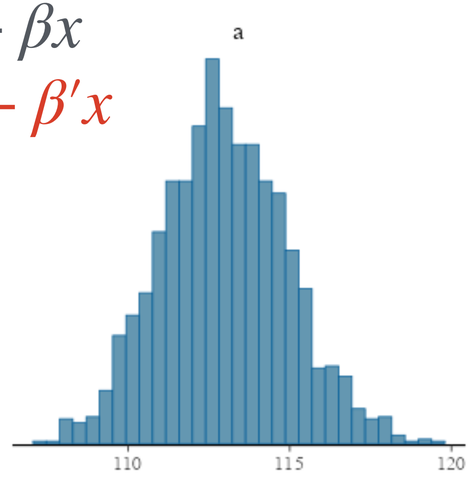


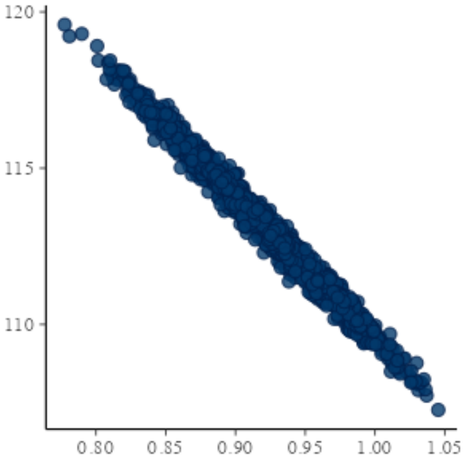


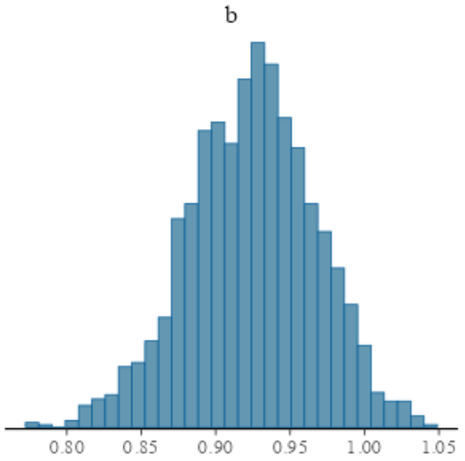


Correlações intercepto-inclinação



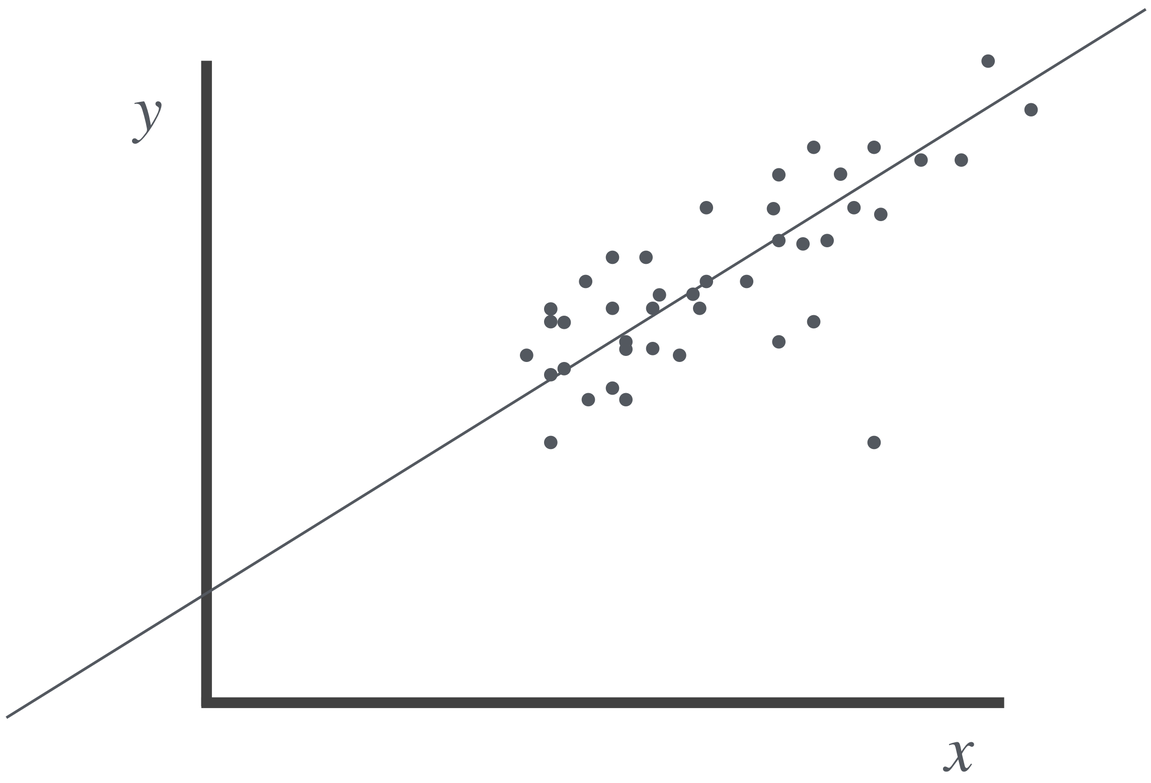






Eliminando as correlações

Modificando o modelo para melhorar a amostragem



$$y = \alpha + \beta x$$

Scaling and shifting parameters can help with convergence

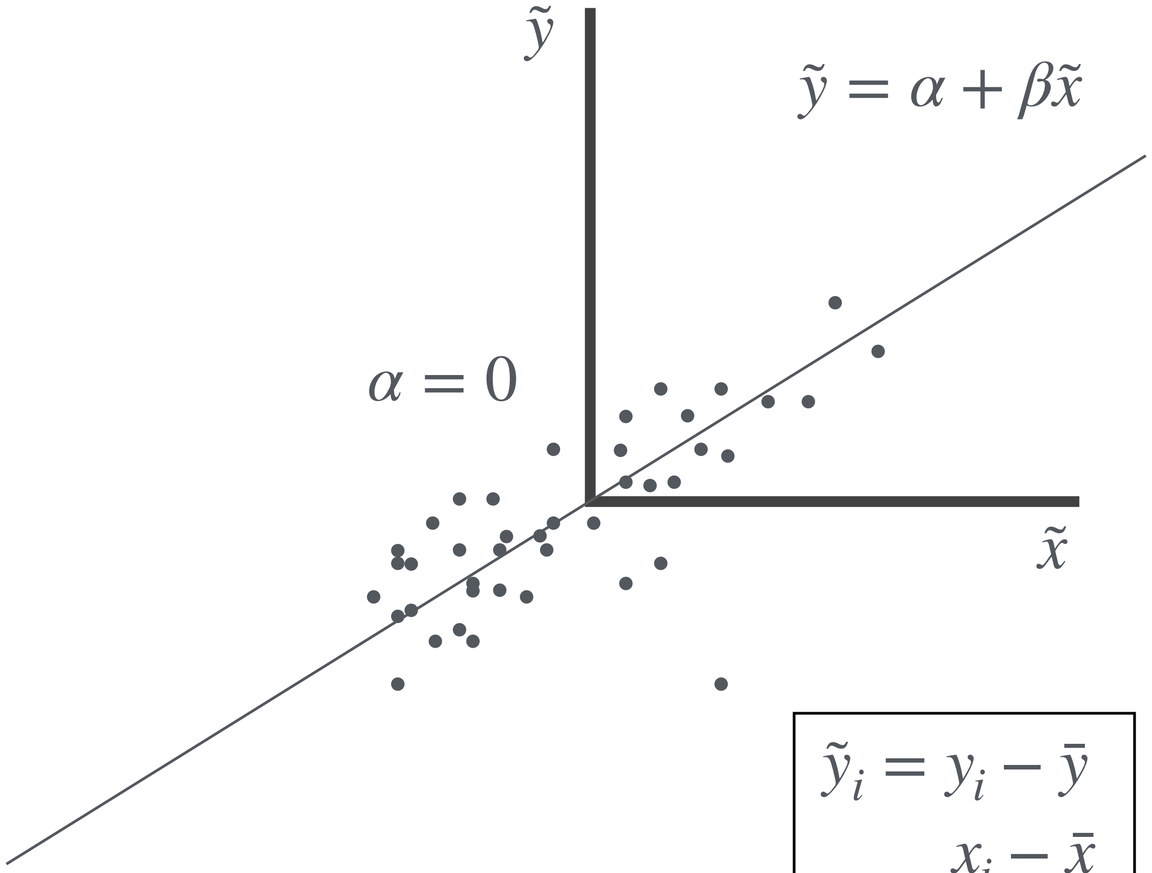
- Centering both x and y values
- Scaling x to unit variance

$$\tilde{y}_i = y_i - \bar{y}$$

$$\tilde{x}_i = \frac{x_i - \bar{x}}{sd(x)}$$

Eliminando as correlações

Modificando o modelo para melhorar a amostragem



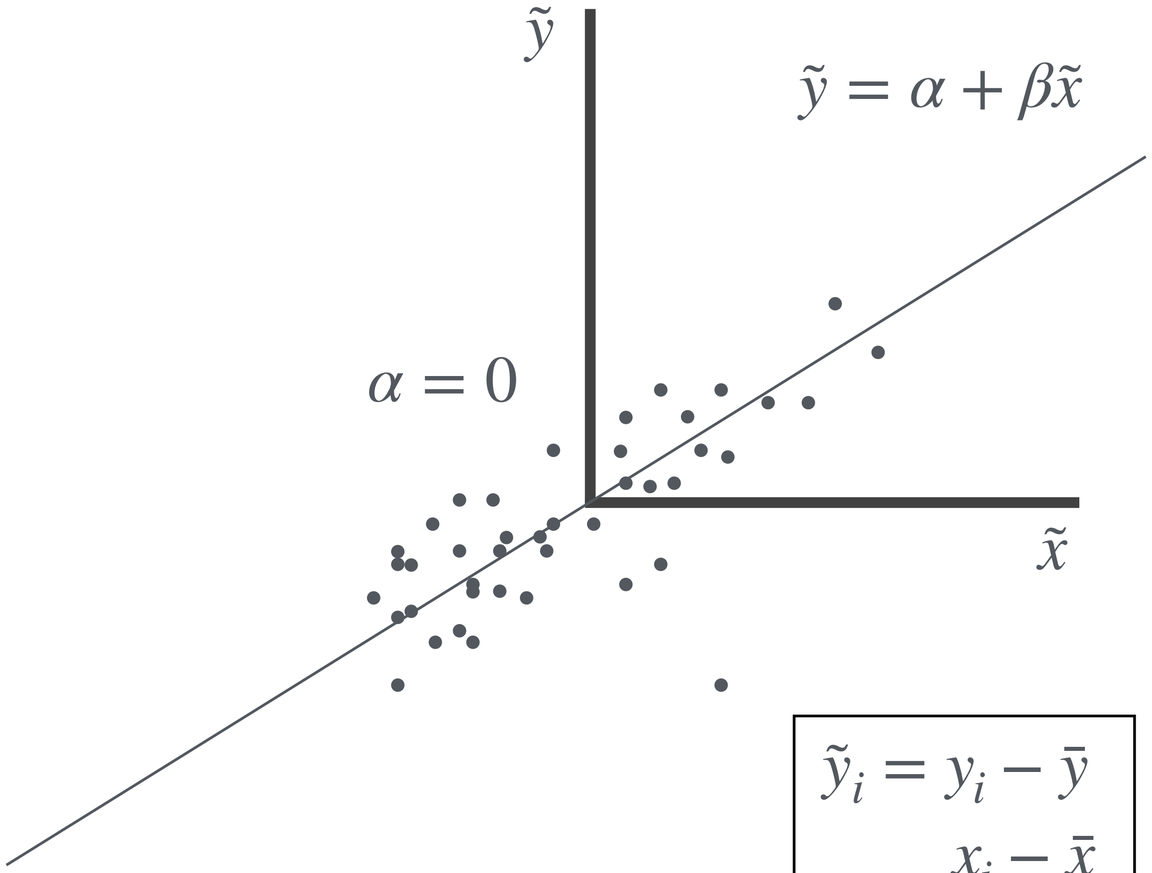
$$\tilde{y}_i = y_i - \bar{y}$$

$$\tilde{x}_i = \frac{x_i - \bar{x}}{sd(x)}$$

```
# Pre-calculate means and sds
mean_x = mean(d2$weight)
sd_x = sd(d2$weight)
mean_y = mean(d2$height)

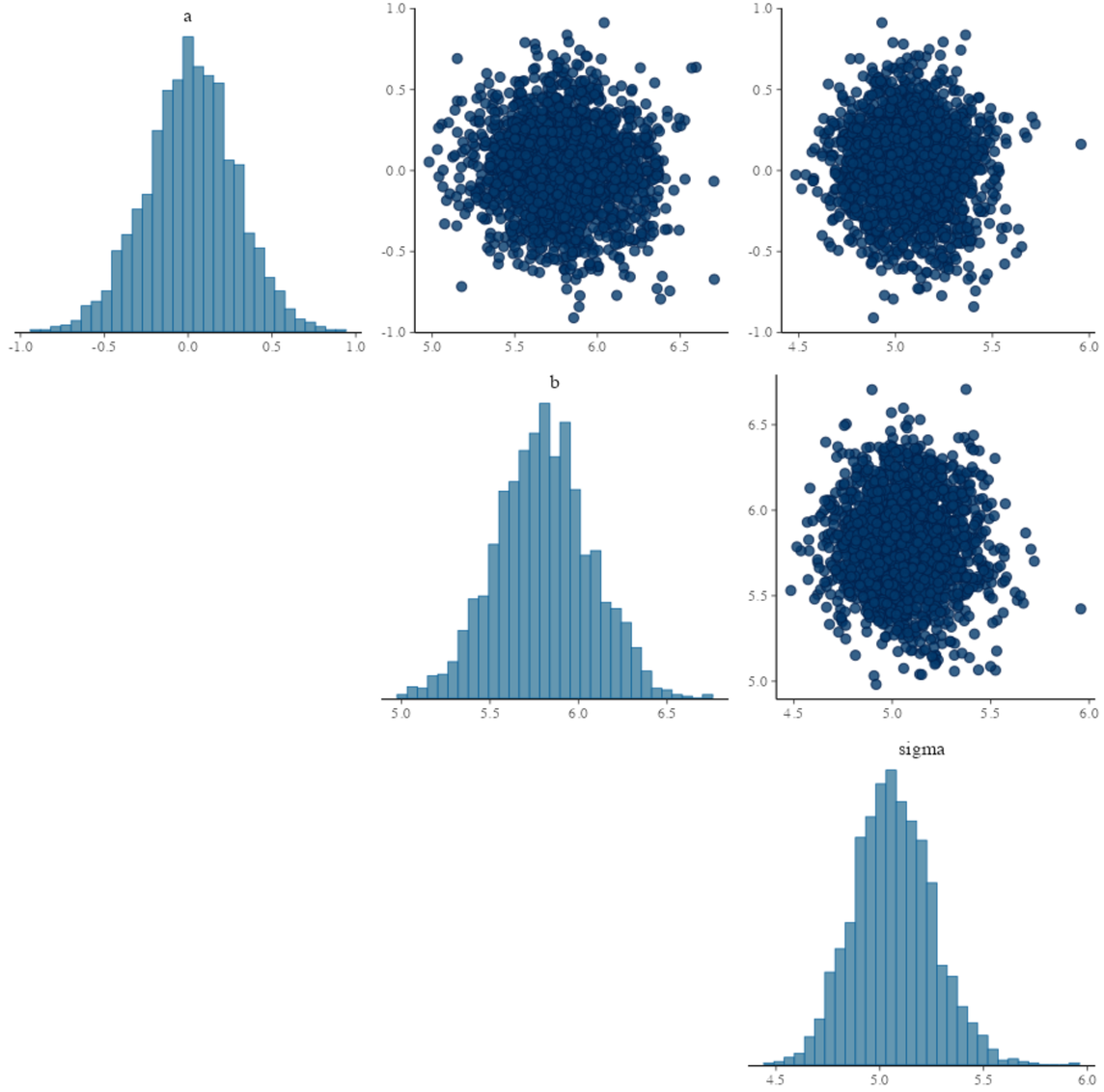
# Model
ulam(alist(
  y ~ normal(mu, sigma),
  mu <- a + b * x,
  a ~ normal(0, 1),
  b ~ lognormal(0, 1),
  sigma ~ exponential(1)),
  data = list(
    y =  d2$height - mean_y,
    x = (d2$weight - mean_x)/sd_x),
  iter = 1000, chains = 4, cores = 4)
```

Eliminando as correlações



$$\tilde{y}_i = y_i - \bar{y}$$

$$\tilde{x}_i = \frac{x_i - \bar{x}}{sd(x)}$$



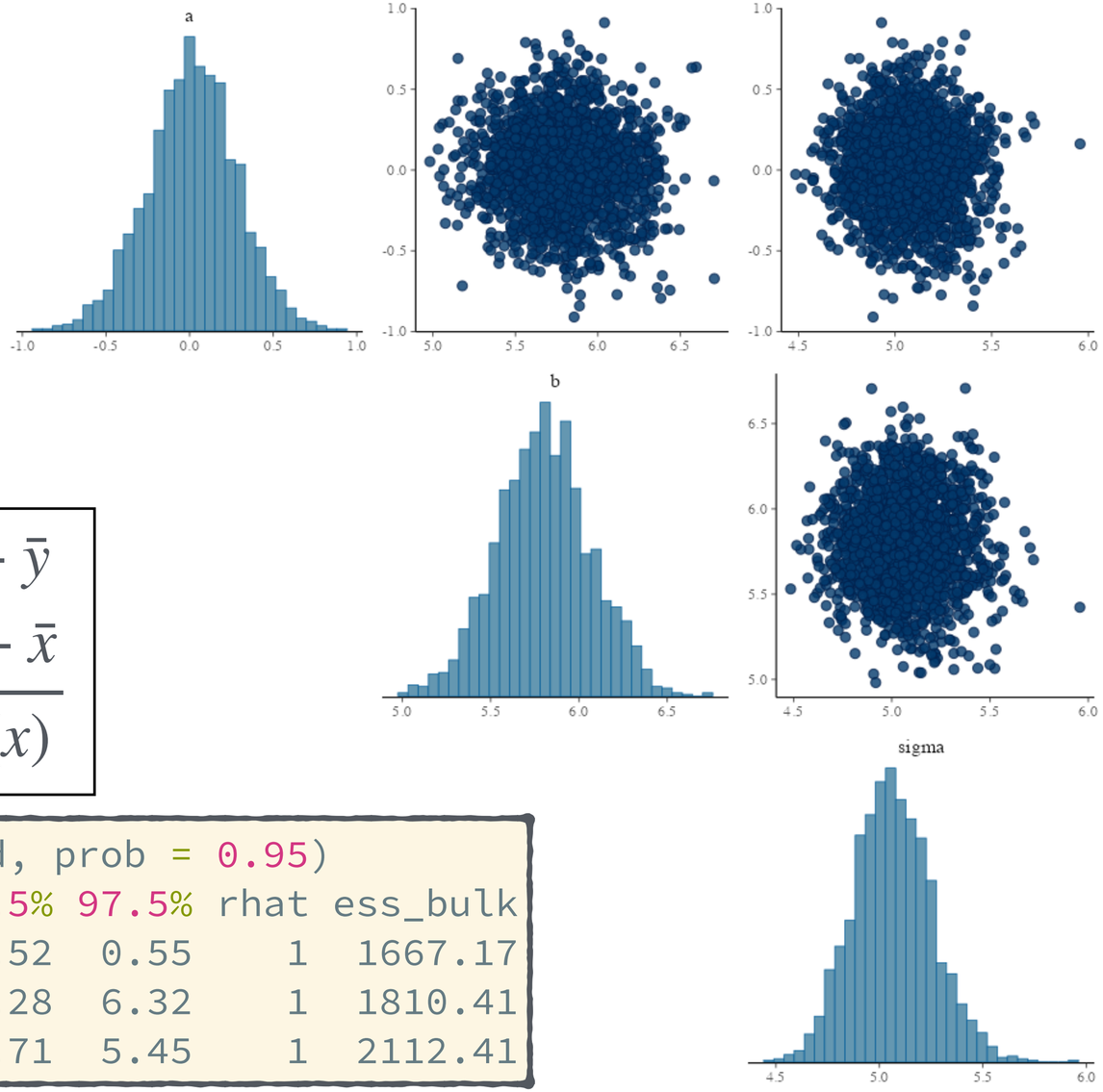
Eliminando as correlações

```
mean_x = mean(d2$weight)
sd_x = sd(d2$weight)
mean_y = mean(d2$height)

# Model
ulam(alist(
  y ~ normal(mu, sigma),
  mu <- a + b * x,
  a ~ normal(0, 1),
  b ~ lognormal(0, 1),
  sigma ~ exponential(1)),
  data = list(
    y =  d2$height - mean_y,
    x = (d2$weight - mean_x)/sd_x),
  iter = 1000, chains = 4, cores = 4)
```

$$\tilde{y}_i = y_i - \bar{y}$$

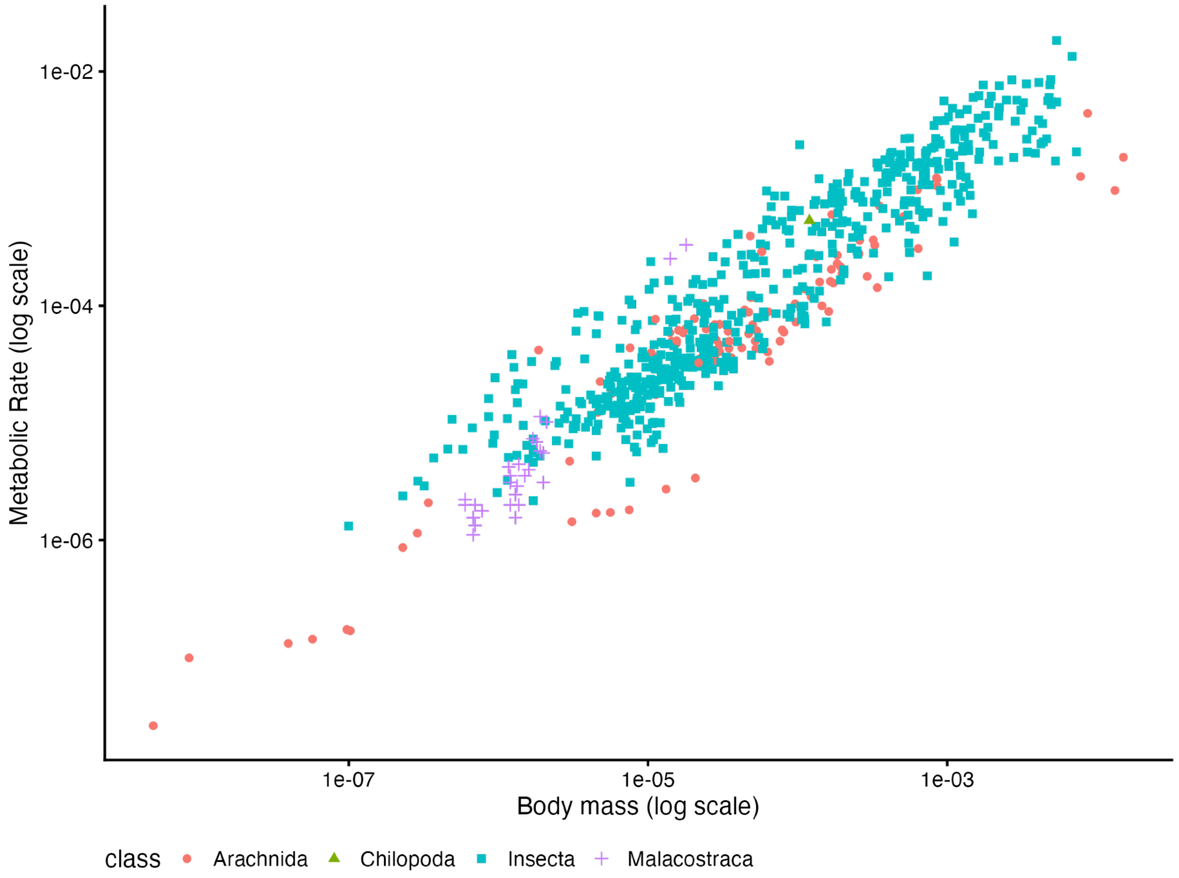
$$\tilde{x}_i = \frac{x_i - \bar{x}}{sd(x)}$$




```
> precis(fit_scaled, prob = 0.95)
       mean   sd  2.5% 97.5% rhat ess_bulk
a      0.02 0.27 -0.52  0.55    1  1667.17
b      5.80 0.27  5.28  6.32    1  1810.41
sigma  5.07 0.19  4.71  5.45    1  2112.41
```

Fazendo perguntas à posteriori

- Podemos usar distribuição a posteriori para responder perguntas sobre os parâmetros
- A amostra da posteriori pode definir intervalos de credibilidade, comparações, testes...
- Exemplo:
  - y: log taxa metabólica
  - x: log massa corporal

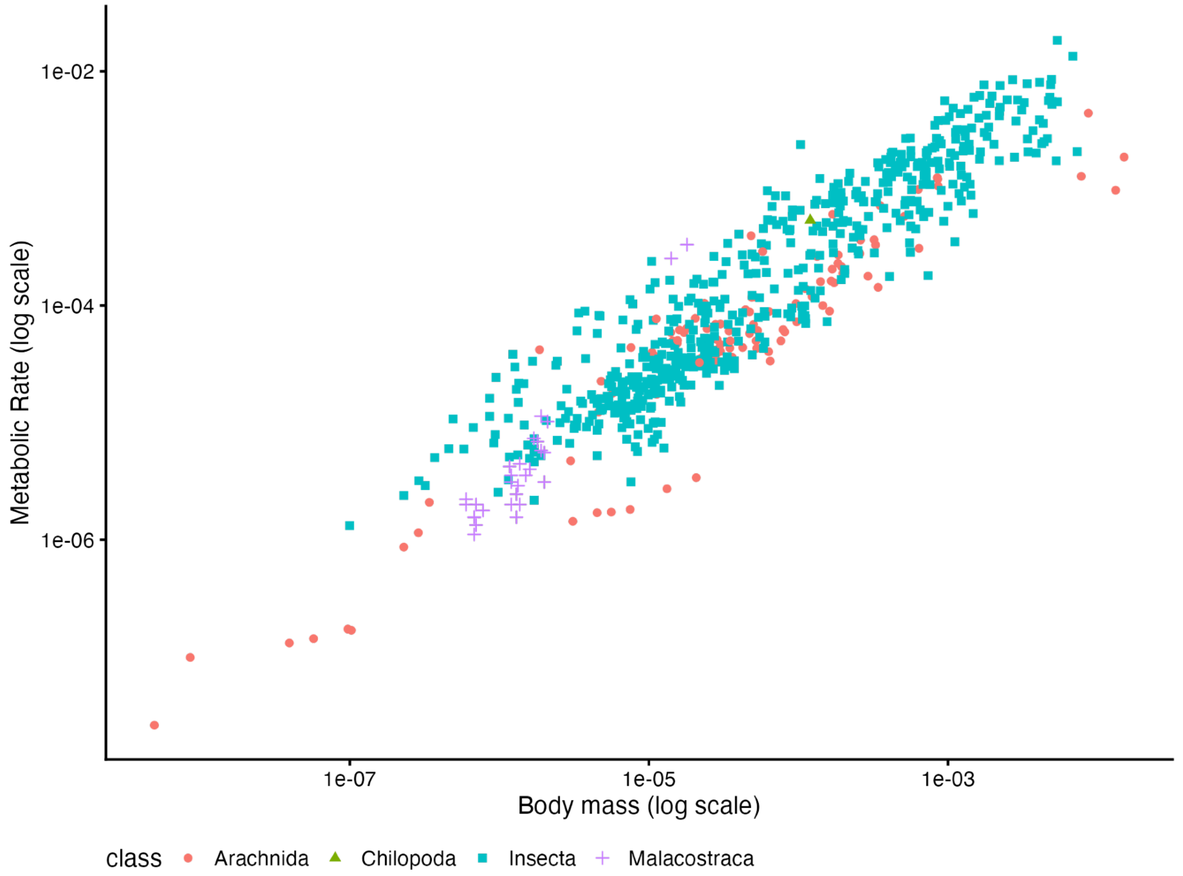


Modelo com uma inclinação por grupo

Uma interação mais simples de codificar (vantagens do rethinking)

```
df = df |>
       mutate(mass = log(body.mass),
              rate = log(metabolic.rate),
              class_i = as.numeric(factor(class)))

m_rate_mass <- ulam(alist(
    rate ~ normal(mu, sigma),
    mu <- a[class_i] + b[class_i] * mass,
    b[class_i] ~ normal(0, 1),
    a[class_i] ~ normal(0, 1),
    sigma ~ exponential(1)
), data = df[, c("mass", "rate", "class_i")],
chains = 4, cores = 4)
```

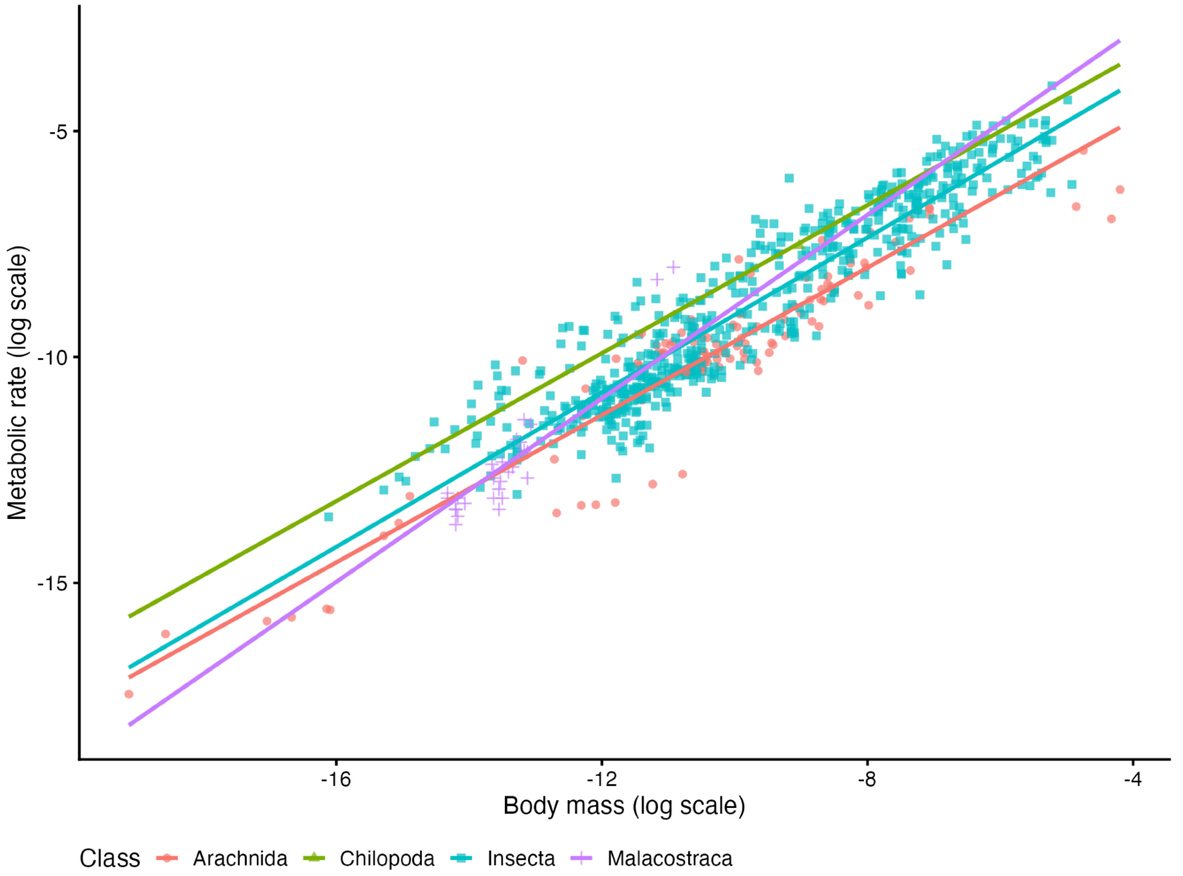


Ajuste do modelo

Uma reta por grupo. As inclinações são diferentes?

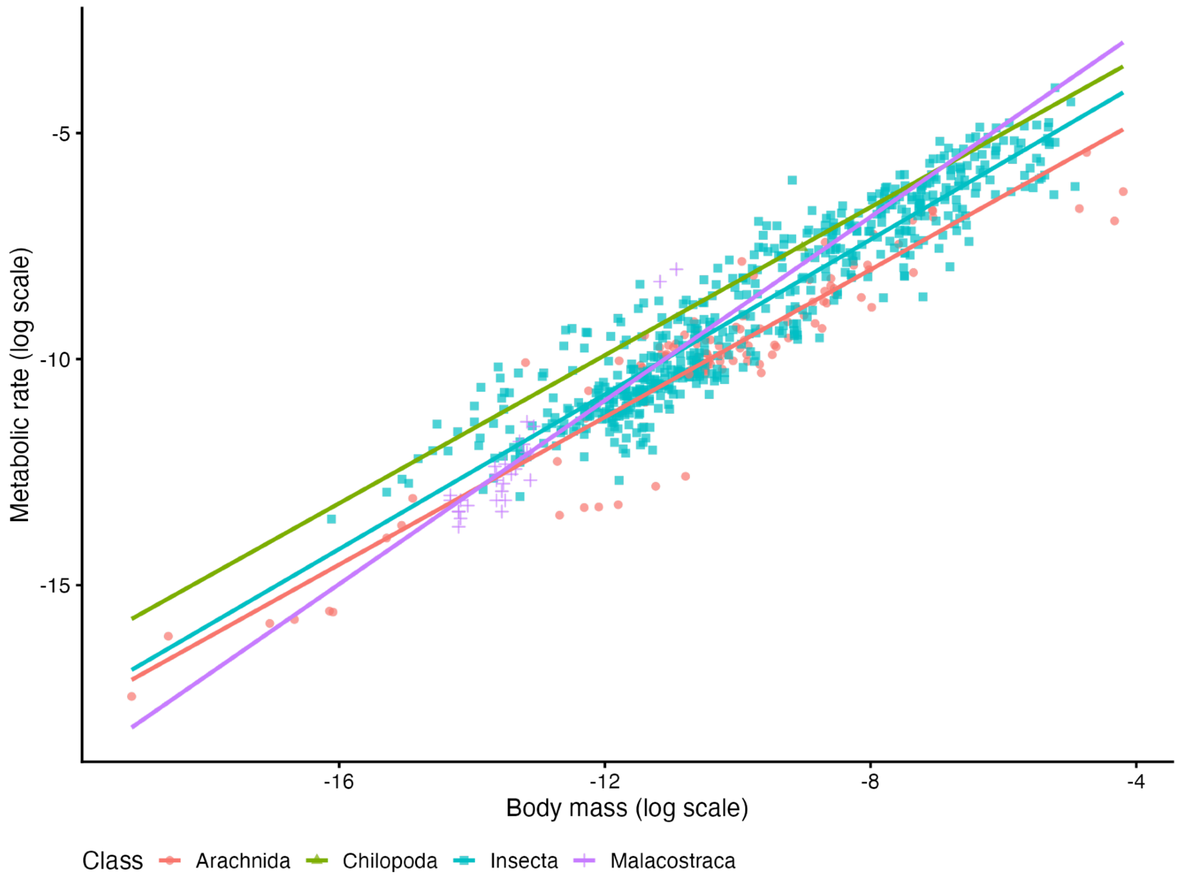
```
df = df |>
       mutate(mass = log(body.mass),
              rate = log(metabolic.rate),
              class_i = as.numeric(factor(class)))

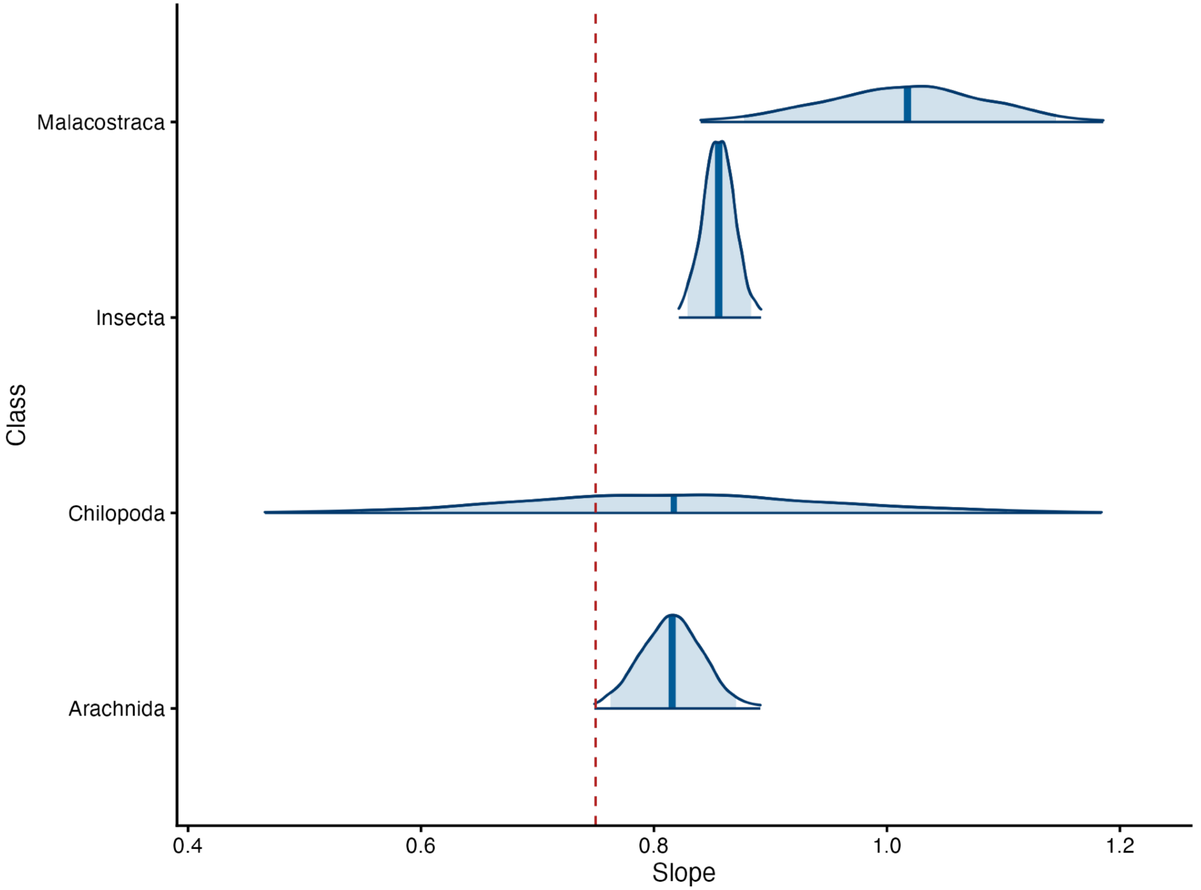
m_rate_mass <- ulam(alist(
    rate ~ normal(mu, sigma),
    mu <- a[class_i] + b[class_i] * mass,
    b[class_i] ~ normal(0, 1),
    a[class_i] ~ normal(0, 1),
    sigma ~ exponential(1)
), data = df[, c("mass", "rate", "class_i")],
chains = 4, cores = 4)
```



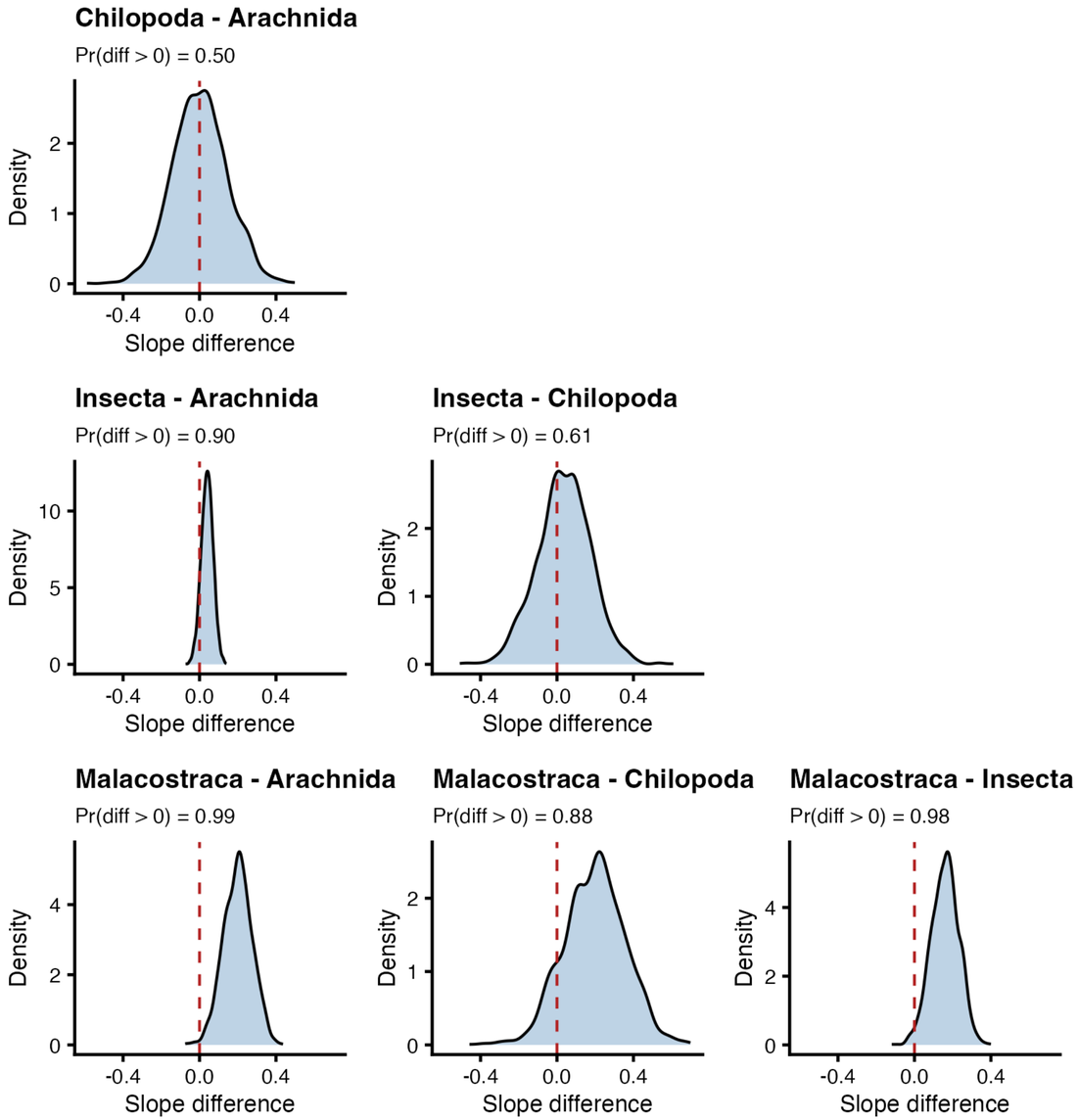
Ajuste do modelo

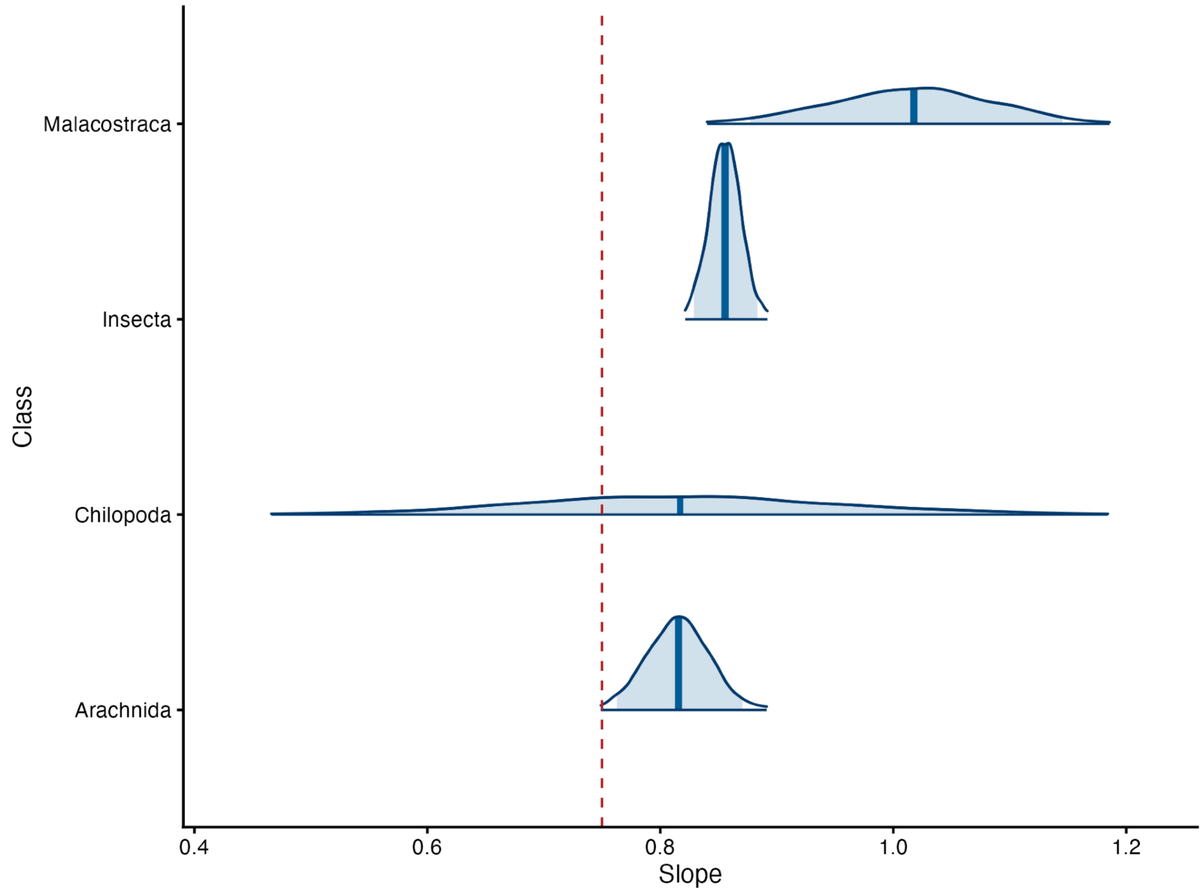
Uma reta por grupo. As inclinações são diferentes?





Comparações par a par das inclinações





Checando o comportamento dos modelos

Depois de ajustar o modelo, podemos usar a posteriori para simular dados sintéticos e comparar esses dados simulados com os dados usados para ajustar o modelo. Discrepâncias podem ser úteis para melhorar o modelo.

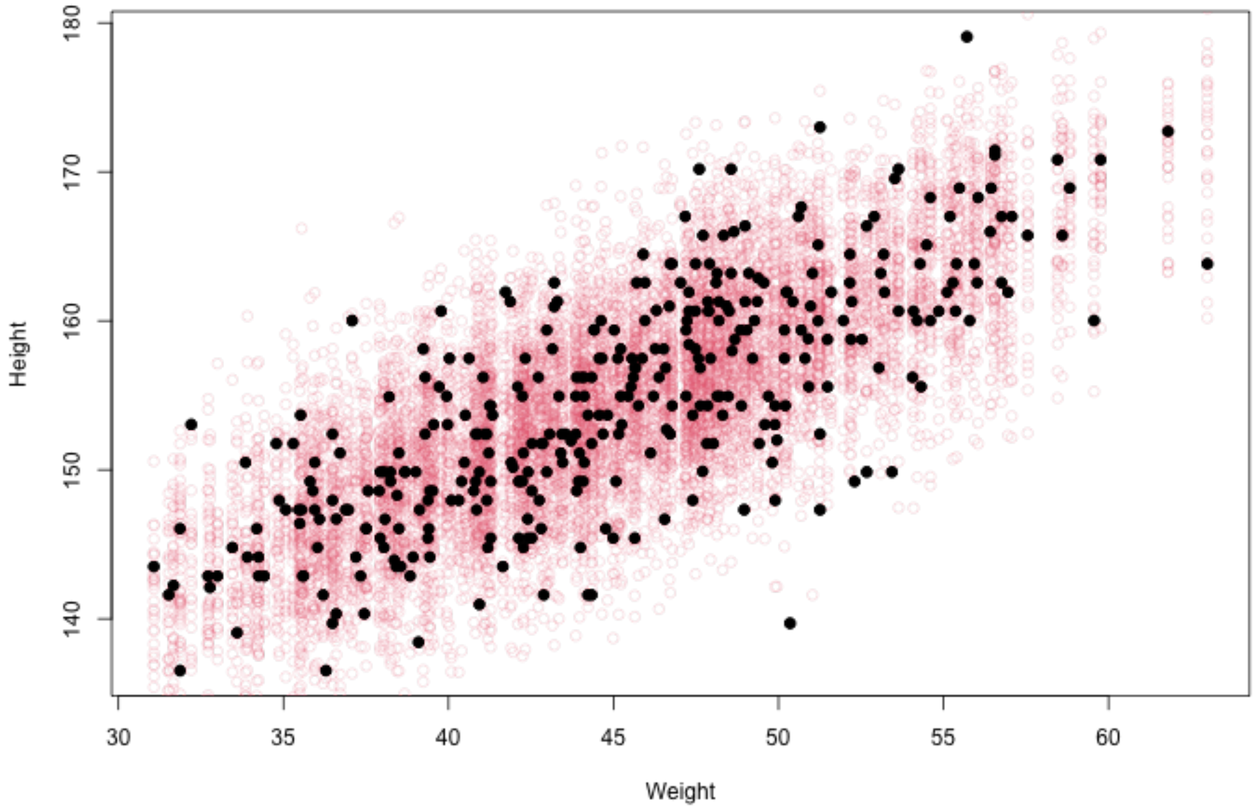
$$y_{sim} \sim P(y_{sim}|y) = \sum_{\theta} P(y_{sim}|\theta)P(\theta|y)$$

Para cada valor dos parâmetros ($\theta_i = \{a_i, b_i, \sigma_i\}$) Podemos simular um conjunto y_{sim} usando a verossimilhança e comparar com os dados observados y.

$$y_{sim} = Normal(a_i + b_i x, \sigma_i)$$

Model Check

Posterior simulations



$$y_{sim} \sim P(y_{sim} \mid y) = \sum_{\theta} P(y_{sim} \mid \theta) P(\theta \mid y)$$

$$y_{sim} = Normal(a_i + b_i x, \sigma_i)$$

Resumo

- A distribuição a posteriori contém muitas informações úteis que não são acessíveis por outros métodos, todas incluindo incerteza.
- Precisamos entender os métodos computacionais que usamos para investigar a distribuição a posteriori.
- As verificações de convergência são fundamentais e podem nos ajudar a diagnosticar modelos ruins.
- O Stan fornece muitas (muitas!) ferramentas para a verificação de modelos.
- Não existem verdadeiros resíduos únicos em modelos bayesianos, mas podemos usar simulações a posteriori para fazer e entender nossas previsões.